

École Polytechnique de Tunis
Université de Carthage
Département Informatique / Génie Électrique

Développement et Implémentation d'un Module de Vision Stéréoscopique Basé sur l'Apprentissage Profond pour la Navigation Autonome d'un Robot Mobile

Mémoire de Master

Auteur :
Boudour Bejaoui

Encadrante :
Dr. Wissal Bey

Année universitaire :
2025 – 2026

14 septembre 2026

Résumé

Les robots mobiles autonomes opèrent de plus en plus dans des environnements complexes et dynamiques où une perception précise et en temps réel est essentielle pour garantir une navigation sûre et fiable. Ces systèmes sont aujourd’hui déployés dans des secteurs aussi variés que la logistique industrielle, l’assistance domestique, la surveillance de sites sensibles et les opérations de sauvetage. Dans ce contexte, la capacité du robot à percevoir et à interpréter son environnement de manière autonome conditionne directement la qualité et la sécurité de ses déplacements.

Le présent mémoire décrit le développement et l’implémentation d’un module de vision stéréoscopique s’appuyant sur l’apprentissage profond pour améliorer les capacités de détection d’obstacles et de navigation autonome d’un robot mobile différentiel de type Turtlebot3. Le système proposé repose sur l’architecture *Pyramid Stereo Matching Network* (PSMNet), un réseau de neurones convolutifs à correspondance stéréo dont la robustesse et la précision sub-pixel ont été démontrées sur les benchmarks de référence. Ce modèle est entraîné sur le jeu de données TartanGround [3], une base de données multimodale à grande échelle comprenant plus de 1,44 million d’échantillons répartis sur 63 environnements de simulation photoréalistes variés, incluant des scènes intérieures, extérieures, urbaines et naturelles. Contrairement aux jeux de données automobiles tels que KITTI, TartanGround a été spécifiquement conçu pour les robots de sol à faible altitude de vue, ce qui le rend particulièrement adapté aux conditions de navigation du Turtlebot3.

Le réseau génère des cartes de disparité denses à partir de paires d’images stéréo rectifiées. Ces cartes sont ensuite converties en cartes de profondeur métriques par la relation géométrique $Z = f \cdot B/d$, puis transformées en nuages de points tridimensionnels pour la segmentation des zones navigables et la délimitation de boîtes englobantes 3D autour des obstacles par les algorithmes RANSAC et DBSCAN. Ce pipeline de perception est intégré dans l’architecture de nœuds ROS (*Robot Operating System*), garantissant une communication temps réel entre les composantes de perception et de navigation.

Un agent d’apprentissage par renforcement, entraîné avec l’algorithme PPO (*Proximal Policy Optimization*), utilise les sorties perceptuelles — carte de profondeur compressée, vecteur d’orientation vers l’objectif et vitesse courante — pour apprendre à planifier des trajectoires sans collision dans des environnements variables simulés sous Gazebo. L’ensemble du pipeline est évalué sur cinquante trajectoires de test couvrant trois scénarios distincts, à l’aide de métriques standardisées incluant l’erreur quadratique de disparité (EPE), le taux de mauvais pixels (D1-all), le taux de succès de navigation et la latence du pipeline.

Mots-clés : vision stéréoscopique, apprentissage profond, réseau de neurones convolutifs, PSMNet, robot mobile, navigation autonome, estimation de disparité, TartanGround, apprentissage par renforcement, ROS, Gazebo, RANSAC, DBSCAN.

Abstract

Autonomous mobile robots increasingly operate in complex and dynamic environments where accurate, real-time perception is essential for safe and reliable navigation. This thesis presents the development and implementation of a stereoscopic vision module leveraging deep learning techniques to enhance obstacle detection and autonomous navigation capabilities of a differential-drive mobile robot (Turtlebot3).

The proposed system combines the Pyramid Stereo Matching Network (PSMNet) architecture, fine-tuned on the TartanGround dataset — a large-scale, multi-modal benchmark comprising over 1.44 million samples across 63 diverse photorealistic simulation environments specifically designed for ground robot perception. The network generates dense disparity maps from rectified stereo image pairs, which are subsequently converted into metric depth maps and three-dimensional point clouds for obstacle detection and navigable zone segmentation using RANSAC ground-plane removal and DBSCAN clustering. The full perception pipeline is deployed as a ROS node operating at 10 Hz.

A reinforcement learning agent trained with the Proximal Policy Optimization (PPO) algorithm uses the perceptual outputs to learn collision-free trajectory planning in variable simulated environments (Gazebo). Experimental evaluations across fifty test trajectories and three distinct scenarios demonstrate significant improvements in obstacle avoidance accuracy and navigation robustness compared to classical stereo-based baselines (SGM).

Keywords : stereo vision, deep learning, convolutional neural network, PSMNet, mobile robot, autonomous navigation, disparity estimation, TartanGround, reinforcement learning, ROS, Gazebo.

Remerciements

Je tiens à exprimer ma sincère gratitude à mon encadrante, Dr. Wissal Bey, pour ses conseils éclairés, son soutien constant et son expertise tout au long de ce travail. Ses remarques constructives et sa disponibilité ont été déterminantes dans l'orientation et la qualité de ce mémoire.

Je remercie également les membres du corps enseignant et les collègues du département pour leurs échanges enrichissants et leur encouragement. Enfin, j'adresse ma profonde reconnaissance à ma famille pour son soutien indéfectible et sa patience tout au long de ce parcours académique.

Table des matières

Résumé	1
Abstract	2
Remerciements	3
Liste des Acronymes	9
1 Introduction	10
1.1 Contexte général	10
1.2 Problématique	11
1.3 Objectifs	12
1.4 Contributions principales	12
1.5 Structure du mémoire	13
2 Revue de Littérature et Fondements Théoriques	14
2.1 Introduction	14
2.2 Vision Stéréo pour l'Estimation de Profondeur	14
2.2.1 Principe de la vision stéréo	14
2.2.2 Méthodes classiques de mise en correspondance stéréo	15
2.2.3 Limitations des méthodes classiques	15
2.3 Apprentissage Profond pour la Mise en Correspondance Stéréo	16
2.3.1 DispNet	16
2.3.2 PSMNet	16
2.3.3 RAFT-Stereo	17
2.4 Réseaux de Neurones Convolutifs en Vision Robotique	17
2.5 Apprentissage par Renforcement pour la Navigation Autonome	18
2.5.1 Fondements de l'apprentissage par renforcement	18
2.5.2 Apprentissage par renforcement profond	18
2.5.3 Applications en robotique	18
2.6 ROS et Gazebo pour le Développement Robotique	19
2.6.1 Robot Operating System (ROS)	19
2.6.2 Environnement de simulation Gazebo	19
2.7 Analyse Comparative	19
2.7.1 Approches de mise en correspondance stéréo	19

2.7.2	Approches de navigation	20
2.8	Discussion	21
2.9	Conclusion	21
3	Module de Perception Stéréoscopique	22
3.1	Introduction	22
3.2	Description du Jeu de Données	22
3.2.1	TartanGround	22
3.2.2	Prétraitement des données	23
3.3	Modèle CNN de Mise en Correspondance Stéréo	24
3.3.1	Sélection du modèle	24
3.3.2	Architecture du réseau	24
3.4	Entraînement du Modèle	25
3.4.1	Procédure d'entraînement	25
3.4.2	Métriques d'évaluation	25
3.5	Génération de Cartes de Disparité	26
3.6	Détection d'Obstacles	26
3.6.1	Détection par analyse de profondeur	26
3.6.2	Détection assistée par CNN	26
3.7	Intégration dans le Pipeline de Navigation	27
3.8	Discussion	27
3.9	Conclusion	28
4	Apprentissage par Renforcement pour la Navigation	29
4.1	Introduction	29
4.2	Formulation du Processus de Décision de Markov	29
4.2.1	Espace d'états	29
4.2.2	Espace d'actions	30
4.2.3	Fonction de récompense	30
4.2.4	Facteur d'actualisation et horizon	30
4.3	Architecture de l'Agent PPO	31
4.3.1	Choix algorithmique	31
4.3.2	Hyperparamètres	31
4.3.3	Architecture du réseau de politique	31
4.4	Stratégie de Curriculum Learning	32
4.5	Environnement d'Entraînement sous ROS 2 / Gazebo	32
4.6	Protocole d'Évaluation sur 50 Trajectoires	32
4.6.1	Conception du protocole	32
4.6.2	Métriques d'évaluation	33
4.6.3	Comparaison avec la baseline classique	33
4.7	Conclusion	33

5	Intégration du Système et Configuration Expérimentale	35
5.1	Introduction	35
5.2	Environnement Matériel et Logiciel	35
5.2.1	Configuration matérielle	35
5.2.2	Environnement logiciel	36
5.2.3	Installation de l'environnement ROS 2 Humble	36
5.3	Structure du Workspace ROS 2	38
5.3.1	Organisation des packages	38
5.3.2	Configuration de la caméra stéréo sur le Turtlebot3 Waffle	39
5.4	Architecture des Nœuds ROS 2	40
5.4.1	Vue d'ensemble du graphe de communication	40
5.4.2	Description des nœuds principaux	41
5.4.3	Topics ROS 2 principaux	42
5.4.4	Nœud PSMNet sous ROS 2 Humble	42
5.4.5	Nœud de l'agent PPO sous ROS 2 Humble	45
5.4.6	Fichier de lancement principal (launch.py)	47
5.5	Configuration de la Simulation Gazebo	50
5.5.1	Mondes Gazebo personnalisés	50
5.5.2	Configuration RViz2 pour la supervision	50
5.5.3	Commandes de lancement de la simulation	51
5.6	Méthodologie d'Évaluation	52
5.6.1	Protocole expérimental	52
5.6.2	Métriques d'évaluation du système complet	53
5.6.3	Comparaison avec la baseline classique	53
5.6.4	Script d'évaluation automatisée	53
5.7	Discussion	55
5.7.1	Choix de ROS 2 Humble vs ROS 1 Noetic	55
5.7.2	Gazebo Classic vs Gazebo Ignition	55
5.8	Conclusion	56
6	Résultats et Analyse	57
7	Conclusion et Perspectives	58

Table des figures

Liste des tableaux

2.1	Comparaison quantitative des méthodes de mise en correspondance stéréo (benchmark KITTI 2015)	19
2.2	Comparaison des stratégies de navigation pour robots mobiles	20
4.1	Composantes du vecteur d'état de l'agent PPO	29
4.2	Hyperparamètres de l'agent PPO	31
4.3	Phases du curriculum learning	32
4.4	Répartition des trajectoires de test par scénario	33
4.5	Métriques d'évaluation du système de navigation autonome	33
5.1	Spécifications matérielles de la station de développement	35
5.2	Spécifications logicielles de l'environnement d'intégration	36
5.3	Nœuds ROS 2 du système et leurs rôles	41
5.4	Topics ROS 2 du système de perception-navigation	42
5.5	Métriques d'évaluation du système de navigation autonome	53

Liste des Acronymes

CNN	Réseau de Neurones Convolutifs (<i>Convolutional Neural Network</i>)
DL	Apprentissage Profond (<i>Deep Learning</i>)
RL	Apprentissage par Renforcement (<i>Reinforcement Learning</i>)
PPO	Optimisation par Politique Proximale (<i>Proximal Policy Optimization</i>)
DQN	Réseau Q Profond (<i>Deep Q-Network</i>)
PSMNet	Réseau Pyramidal de Correspondance Stéréo (<i>Pyramid Stereo Matching Network</i>)
SPP	Pooling Spatial Pyramidal (<i>Spatial Pyramid Pooling</i>)
ROS	Robot Operating System
RGB	Rouge Vert Bleu (<i>Red Green Blue</i>)
SLAM	Localisation et Cartographie Simultanées (<i>Simultaneous Localization and Mapping</i>)
EPE	Erreur de Point Final (<i>End-Point Error</i>)
GPU	Processeur Graphique (<i>Graphics Processing Unit</i>)
BEV	Vue Aérienne (<i>Bird's-Eye View</i>)
DBSCAN	Regroupement Spatial Basé sur la Densité (<i>Density-Based Spatial Clustering of Applications with Noise</i>)
RANSAC	Consensus d'Échantillon Aléatoire (<i>Random Sample Consensus</i>)
IMU	Unité de Mesure Inertielle (<i>Inertial Measurement Unit</i>)
LiDAR	Détection et Télémétrie Laser (<i>Light Detection and Ranging</i>)
ONNX	Échange Ouvert de Réseaux de Neurones (<i>Open Neural Network Exchange</i>)
FPS	Images par Seconde (<i>Frames Per Second</i>)
SGM	Correspondance Semi-Globale (<i>Semi-Global Matching</i>)
MDP	Processus de Décision de Markov (<i>Markov Decision Process</i>)
BM	Correspondance par Blocs (<i>Block Matching</i>)

Chapitre 1

Introduction

1.1 Contexte général

La dernière décennie a été marquée par un essor sans précédent du déploiement de robots mobiles autonomes dans un large éventail d'applications réelles. De la logistique industrielle et l'automatisation des entrepôts à l'assistance domestique et aux opérations de recherche et de sauvetage, les robots sont appelés à opérer avec des niveaux croissants d'indépendance dans des environnements non structurés, dynamiques et souvent imprévisibles [1]. Au cœur de tout système de navigation autonome se trouve la capacité à percevoir l'environnement avec précision, fiabilité et rapidité suffisante pour soutenir la prise de décision en temps réel.

Les approches traditionnelles de perception robotique ont longtemps reposé sur des modalités de capteurs classiques tels que les télémètres laser (LiDAR), les capteurs ultrasoniques et les caméras monoculaires. Si ces technologies ont permis des progrès considérables, chacune présente des limitations inhérentes : les systèmes LiDAR sont coûteux et produisent des nuages de points épars à grande distance ; les capteurs ultrasoniques offrent une faible résolution angulaire ; et les caméras monoculaires, bien qu'économiques et riches en informations texturales, sont fondamentalement incapables de récupérer directement une profondeur métrique à partir d'un seul point de vue.

La vision stéréoscopique — l'utilisation de deux caméras latéralement déplacées pour inférer la profondeur par triangulation — offre un juste milieu convaincant. En exploitant la disparité binoculaire, les systèmes stéréo fournissent des estimations de profondeur denses et métriques à une fraction du coût du LiDAR, ce qui les rend particulièrement attrayants pour les plateformes robotiques à ressources limitées. Cependant, les algorithmes classiques de mise en correspondance stéréo tels que le Semi-Global Matching (SGM) et le Block Matching (BM) restent fragiles en présence de surfaces sans texture, de matériaux réfléchissants, d'occultations et de conditions d'éclairage variables, situations courantes dans les déploiements réels.

L'émergence de l'apprentissage profond, et des réseaux de neurones convolutifs en particulier, a fondamentalement transformé le paysage de la vision par ordinateur. Les architectures entraînables de bout en bout telles que DispNet [6], PSMNet [4] et RAFT-Stereo [5] ont démontré que les représentations de caractéristiques apprises peuvent substantiellement surpasser les coûts de mise en correspondance artisanaux, atteignant une précision remarquable sur les benchmarks standards tels que KITTI [7] et Middlebury [8]. Pourtant, la transposition de ces avancées à la navigation des robots de sol — où le point de vue est bas, la géométrie de la scène est dominée par des plans de sol, et le robot doit opérer dans des environnements éloignés des jeux de données automobiles —

demeure un défi ouvert et peu exploré [3].

C'est dans cette problématique que s'inscrit le présent travail. En exploitant la complémentarité de la stéréovision et de l'apprentissage profond, ce mémoire vise à proposer un système de navigation autonome robuste, adapté aux caractéristiques spécifiques des robots mobiles de sol évoluant en environnement intérieur et extérieur.

1.2 Problématique

Malgré les progrès réalisés dans l'estimation de profondeur stéréo pour la conduite autonome, un écart significatif persiste entre les capacités démontrées sur les benchmarks de conduite et les exigences des robots mobiles de sol. Plusieurs problèmes fondamentaux motivent le présent travail :

1. **Inadéquation de domaine** : Les jeux de données stéréo existants tels que KITTI ont été collectés à partir de caméras montées sur des véhicules à une hauteur d'environ 1,6 m, orientées vers l'avant le long des chaussées. Les robots de sol — qu'ils soient à roues, à entraînement différentiel ou à pattes — opèrent à des points de vue substantiellement plus bas et traversent des terrains très diversifiés incluant des couloirs intérieurs, des prairies et des places urbaines. Les modèles entraînés sur des données de conduite ne se généralisent pas bien à ces conditions [3]. Cette inadéquation de domaine se traduit concrètement par une dégradation de l'erreur de point final (EPE) pouvant atteindre deux à trois fois la valeur observée sur le domaine source, rendant les prédictions inutilisables pour une navigation sécurisée.
2. **Détection d'obstacles à courte et moyenne portée** : La navigation autonome des robots de sol exige une détection fiable des obstacles à des distances de 0,5 m à 10 m, une plage où les valeurs de disparité sont élevées et les exigences de précision sont les plus sévères. Des cartes de disparité denses avec une précision sub-pixel sont nécessaires pour distinguer les surfaces navigables des obstacles de faible hauteur tels que des seuils de porte, des câbles au sol ou des irrégularités de surface. Les méthodes classiques comme SGM atteignent ici leurs limites, notamment dans les zones texturées uniformément (murs peints, sols lisses) et sous éclairage artificiel variable.
3. **Intégration avec la navigation** : La perception de profondeur n'est qu'un moyen et non une fin en soi. Pour qu'un robot navigue de manière autonome, la sortie perceptuelle doit être efficacement fusionnée avec un mécanisme de planification capable de générer des trajectoires sûres et optimales en temps réel. Les planificateurs de chemin classiques opèrent sur des cartes simplifiées et peinent dans les scènes hautement dynamiques. L'apprentissage par renforcement offre une alternative axée sur les données qui peut s'adapter à la variabilité environnementale par essais et erreurs, et dont la convergence peut être accélérée par l'intégration des informations 3D issues de la stéréovision.

Ces considérations motivent collectivement le développement d'un pipeline de perception et de navigation intégré qui combine la mise en correspondance stéréo profonde, entraînée sur des

données spécifiques aux robots de sol, avec un agent d'apprentissage par renforcement capable d'exploiter la compréhension tridimensionnelle de la scène résultante pour une navigation autonome robuste.

1.3 Objectifs

L'objectif principal de ce mémoire est de concevoir, d'implémenter et d'évaluer un système complet de perception stéréoscopique et de navigation autonome pour un robot mobile, avec l'apprentissage profond en son cœur. Plus précisément, le travail vise à :

1. Mener une revue systématique de la littérature sur les méthodes de vision stéréo basées sur l'apprentissage profond, les pipelines de perception des robots mobiles, et les approches d'apprentissage par renforcement pour la navigation autonome.
2. Développer un module de mise en correspondance stéréo basé sur les CNN — fondé sur l'architecture PSMNet — entraîné et évalué sur le jeu de données TartanGround [3], un jeu de données multimodal à grande échelle spécifiquement conçu pour la perception des robots de sol dans 63 environnements photoréalistes diversifiés incluant des scènes intérieures avec robots différentiels, ce qui constitue une réponse directe au problème d'inadéquation de domaine identifié.
3. Implémenter un pipeline de conversion disparité-profondeur et un module de détection d'obstacles tridimensionnels capable de segmenter les zones navigables par élimination du plan du sol (RANSAC) et de générer des boîtes englobantes 3D autour des obstacles par regroupement spatial (DBSCAN), le tout en temps réel au sein de nœuds ROS.
4. Concevoir et entraîner un agent d'apprentissage par renforcement utilisant l'algorithme PPO qui consomme les sorties perceptuelles — carte de profondeur compressée 64×64 , vecteur d'orientation vers l'objectif, vitesse linéaire et angulaire courantes — pour apprendre des trajectoires optimales sans collision dans des environnements simulés.
5. Intégrer le pipeline complet au sein du framework de simulation ROS/Gazebo avec le Turtlebot3 Waffle et conduire des évaluations systématiques de performance sur trois scénarios de test distincts (couloir, espace ouvert, entrepôt complexe).
6. Quantifier la contribution de la perception stéréoscopique basée sur l'apprentissage profond par rapport aux baselines stéréo classiques en termes de précision de profondeur (EPE, D1-all), taux de détection d'obstacles, taux de succès de navigation et latence du pipeline.

1.4 Contributions principales

Les contributions principales de ce mémoire sont au nombre de trois :

- **Un module de perception stéréo optimisé pour les robots de sol** : Nous adaptons et affinons (*fine-tunons*) PSMNet sur TartanGround, démontrant une meilleure généralisation aux scénarios à faible point de vue et terrains diversifiés par rapport aux modèles entraînés

exclusivement sur des données automobiles. Le choix du sous-ensemble *diff* (robot différentiel) et des environnements intérieurs du jeu de données TartanGround assure une adéquation maximale avec les conditions réelles d'utilisation du Turtlebot3.

- **Un pipeline perception-navigation de bout en bout** : Nous présentons un système étroitement intégré qui connecte l'estimation de profondeur stéréo, la segmentation d'obstacles 3D (RANSAC + DBSCAN) et la planification de trajectoire basée sur l'apprentissage par renforcement (PPO) au sein d'une architecture compatible ROS opérant à 10 Hz. Ce pipeline bout en bout, combinant vision stéréo profonde et navigation par renforcement, constitue — à notre connaissance — une approche originale non encore réalisée pour le Turtlebot3 dans des environnements dynamiques variés.
- **Une évaluation comparative systématique** : Nous fournissons des preuves quantitatives de la supériorité de la mise en correspondance stéréo basée sur l'apprentissage profond par rapport aux approches classiques SGM/BM dans le contexte de la navigation des robots de sol, en utilisant des métriques standardisées (EPE, D1-all, Bad2.0, Bad3.0) sur le benchmark TartanGround et des scénarios de simulation Gazebo personnalisés.

1.5 Structure du mémoire

Le reste de ce mémoire est organisé comme suit :

- **Chapitre 2** présente une revue de littérature complète couvrant la vision stéréo, l'apprentissage profond pour l'estimation de profondeur, et l'apprentissage par renforcement pour la navigation robotique.
- **Chapitre 3** décrit le module de perception stéréoscopique : le jeu de données TartanGround, l'architecture PSMNet, la procédure d'entraînement et le pipeline de détection d'obstacles 3D.
- **Chapitre 4** détaille l'agent d'apprentissage par renforcement, la formulation de l'environnement dans ROS/Gazebo et la stratégie d'entraînement pour la navigation autonome.
- **Chapitre 5** présente l'intégration complète du système, la configuration expérimentale et les scénarios d'évaluation.
- **Chapitre 6** rapporte et analyse les résultats expérimentaux, incluant des métriques quantitatives et des visualisations qualitatives.
- **Chapitre 7** résume les conclusions principales, discute des limitations de l'approche et propose des pistes de recherche futures.

Chapitre 2

Revue de Littérature et Fondements Théoriques

2.1 Introduction

La navigation autonome en robotique mobile repose sur la capacité d'un robot à percevoir, comprendre et interagir avec son environnement en temps réel. Cette capacité exige la convergence de plusieurs disciplines : la vision par ordinateur pour l'acquisition et le traitement des données visuelles, l'apprentissage profond pour l'extraction de représentations sémantiques riches, et l'apprentissage par renforcement pour la prise de décision adaptative. Les avancées récentes dans ces domaines ont permis des améliorations substantielles des systèmes de perception et de planification de trajectoire.

Le présent chapitre expose les fondements théoriques nécessaires à la compréhension du travail proposé. Il traite successivement de la vision stéréo pour l'estimation de profondeur, des réseaux de neurones convolutifs pour la perception visuelle, des méthodes d'apprentissage profond pour la mise en correspondance stéréo, de l'apprentissage par renforcement pour la navigation, et des frameworks ROS et Gazebo comme environnements de développement. Une analyse comparative positionne enfin l'approche proposée par rapport aux travaux existants.

2.2 Vision Stéréo pour l'Estimation de Profondeur

2.2.1 Principe de la vision stéréo

La vision stéréo est une technique d'estimation de profondeur exploitant la relation géométrique entre deux caméras dont les axes optiques sont parallèles et séparés par une distance B appelée *baseline*. En capturant deux images du même objet depuis des points de vue légèrement différents, il est possible de calculer la disparité entre les pixels correspondants et d'en déduire la distance aux objets de la scène. Ce principe biomimétique reproduit le fonctionnement de la vision binoculaire humaine, qui exploite la parallaxe entre les deux yeux pour percevoir la profondeur.

La profondeur Z est calculée à partir de la disparité d par la relation :

$$Z = \frac{f \cdot B}{d} \quad (2.1)$$

où f est la longueur focale de la caméra (exprimée en pixels), B est la distance entre les centres optiques des deux caméras (la baseline, en mètres), et d est la disparité mesurée en pixels entre les

projections du même point 3D dans les deux images. Cette relation fondamentale montre que la précision de l'estimation de profondeur est inversement proportionnelle à la disparité : les objets proches génèrent de grandes disparités (estimation plus précise), tandis que les objets lointains génèrent de petites disparités (estimation moins précise). Pour le Turtlebot3, la plage de distances pertinente est de 0,5 m à 10 m, ce qui correspond à des disparités comprises entre environ 5 et 100 pixels selon la configuration de la caméra.

La mise en correspondance stéréo, étape clé du processus, consiste à identifier pour chaque pixel de l'image gauche le pixel correspondant dans l'image droite. Elle s'appuie sur la contrainte épipolaire : si les images sont rectifiées, les pixels correspondants se trouvent sur la même ligne horizontale, réduisant le problème de recherche bidimensionnel en un problème unidimensionnel [11].

2.2.2 Méthodes classiques de mise en correspondance stéréo

Les approches traditionnelles de mise en correspondance stéréo peuvent être regroupées en deux grandes familles :

- **Correspondance par blocs (BM)** : Cette méthode compare des fenêtres locales de pixels entre les deux images en utilisant des métriques de similarité telles que la somme des différences absolues (SAD) ou la somme des différences au carré (SSD). Elle est rapide (opérable en temps réel, ~ 20 ms) mais très sensible aux zones sans texture, aux occultations et aux surfaces répétitives typiques des environnements intérieurs (murs peints, sols carrelés).
- **Correspondance semi-globale (SGM)** : Proposée par Hirschmüller [7], cette méthode introduit une régularisation spatiale par agrégation de coût sur plusieurs directions. Elle produit des cartes de disparité plus denses et plus cohérentes que BM, avec une précision nettement supérieure ($EPE \approx 3,5$ px sur KITTI), au prix d'un temps de calcul plus élevé (~ 50 ms). SGM constitue la baseline de référence dans ce travail.

2.2.3 Limitations des méthodes classiques

Les méthodes classiques présentent plusieurs limitations rédhibitoires pour une utilisation en conditions réelles :

- **Sensibilité au bruit** : Les variations d'illumination, les reflets spéculaires et le bruit du capteur dégradent fortement les métriques de similarité.
- **Mauvaise généralisation** : Les paramètres (taille de fenêtre, pénalités de régularisation) sont optimisés pour un contexte spécifique et ne se transfèrent pas bien à des environnements différents.
- **Précision limitée** : Dans les zones sans texture (murs uniformes, sols lisses), la mise en correspondance est ambiguë, ce qui produit des régions de disparité invalide pouvant masquer des obstacles bas ou des irrégularités de surface essentielles à la navigation sécurisée du Turtlebot3.

Ces limitations ont motivé l'adoption d'approches basées sur l'apprentissage profond, capables d'apprendre des représentations discriminantes robustes à ces conditions défavorables.

2.3 Apprentissage Profond pour la Mise en Correspondance Stéréo

Les méthodes d'apprentissage profond ont transformé la vision stéréo en apprenant des représentations de caractéristiques directement à partir des données, sans recourir à des métriques de similarité artisanales. Ces approches end-to-end permettent une estimation plus robuste dans des conditions variées et atteignent des précisions sub-pixel.

2.3.1 DispNet

DispNet [6] a été l'un des premiers réseaux de neurones convolutifs end-to-end pour l'estimation de disparité. Son architecture encodeur-décodeur traite les paires d'images stéréo concaténées et produit une carte de disparité dense en une seule passe.

Architecture : L'encodeur extrait des caractéristiques hiérarchiques à différentes résolutions par des convolutions successives avec sous-échantillonnage. Le décodeur remonte en résolution par des convolutions transposées (upsampling) et des connexions de saut. Une couche de corrélation 1D entre les deux flux de caractéristiques fournit la mesure de similarité entre les pixels.

Avantages et limites :

- + Architecture simple et inférence rapide (~ 60 ms sur GPU)
- + Première démonstration de la faisabilité d'une approche end-to-end
- Précision modérée ($EPE \approx 1,8$ px sur KITTI)
- Performances dégradées dans les scènes complexes avec de nombreuses occultations

2.3.2 PSMNet

PSMNet (*Pyramid Stereo Matching Network*) [4] constitue une avancée majeure en introduisant un module de pooling spatial pyramidal (SPP) et un volume de coût 4D agrégé par des convolutions 3D. Cette architecture, présentée à la conférence CVPR 2018, reste un choix de référence pour les applications robotiques en raison de son excellent rapport précision/complexité.

Architecture en quatre étapes :

1. **Extraction de caractéristiques :** Un réseau CNN partagé (mêmes poids pour les images gauche et droite) extrait des caractéristiques à la résolution $H8 \times W8$. Le module SPP capture le contexte à quatre échelles de pooling (8, 16, 32, 64), permettant d'intégrer des informations locales et globales.
2. **Construction du volume de coût :** Les caractéristiques des deux images sont concaténées canal par canal pour chaque niveau de disparité $d \in [0, D_{\max}-1]$, formant un volume 4D de dimensions $B \times 2C \times D \times H8 \times W8$. Ce volume encode la similarité entre chaque pixel de l'image gauche et ses correspondants potentiels dans l'image droite.

3. **Agrégation et régression de disparité** : Des convolutions 3D en structure hourglass (encodeur-décodeur 3D) régularisent spatialement le volume de coût. La disparité finale est obtenue par softargmax, une opération différentiable qui préserve la précision sub-pixel : $\hat{d} = \arg\max_d d \cdot \sigma - c_d$.
4. **Raffinement** : Une supervision multi-échelle pendant l'entraînement ($\mathcal{L} = 0,5\mathcal{L}_1 + 0,7\mathcal{L}_2 + 1,0\mathcal{L}_3$) guide l'apprentissage à plusieurs niveaux de résolution.

Avantages et limites :

- + Haute précision : EPE $\approx 1,1$ px, D1-all $\approx 2,3$ % sur KITTI
- + Code officiel disponible (GitHub, 3 000+ étoiles), poids pré-entraînés sur SceneFlow
- + Fine-tuning accessible : adapté au cadre d'un mémoire de master
- Latence d'environ 180 ms par image, nécessitant une optimisation pour ROS
- Empreinte mémoire GPU significative lors de l'entraînement en batch

Choix pour ce travail : PSMNet est retenu comme architecture principale en raison de son rapport optimal entre précision et accessibilité de mise en œuvre. L'accent mis sur le fine-tuning sur TartanGround (environnements de sol intérieurs) permettra de dépasser les performances obtenues sur KITTI tout en restant dans des délais d'entraînement raisonnables.

2.3.3 RAFT-Stereo

RAFT-Stereo [5] est une méthode de pointe basée sur le raffinement itératif par unités récurrentes (GRU). Héritant de l'architecture RAFT pour le flux optique, elle maintient et met à jour itérativement un champ de corrélation dense entre les caractéristiques des deux images.

Avantages et limites :

- + Très haute précision : EPE $\approx 0,7$ px, D1-all $\approx 1,6$ %
- + Forte généralisation inter-domaines
- Implémentation complexe, coût computationnel élevé
- Moins adapté aux contraintes temps réel d'un robot embarqué

2.4 Réseaux de Neurones Convolutifs en Vision Robotique

Les CNN constituent le fondement de la plupart des systèmes de vision modernes pour la robotique. Leur capacité à extraire automatiquement des représentations hiérarchiques — des bords et textures de bas niveau aux structures sémantiques de haut niveau — les rend indispensables pour les tâches de détection d'objets, classification d'images et segmentation sémantique.

En robotique mobile, les architectures légères sont souvent préférées pour garantir des performances temps réel sur des plateformes embarquées. La conception architecturale doit équilibrer précision et latence : un modèle trop lourd ne peut opérer qu'à 2 FPS, insuffisant pour une réaction fluide aux obstacles dynamiques, tandis qu'un modèle trop simplifié produira des cartes de profondeur dégradées. Dans ce travail, PSMNet fine-tuné et exporté en ONNX vise une latence inférieure à 100 ms (10+ FPS), compatible avec la boucle de navigation ROS à 10 Hz.

2.5 Apprentissage par Renforcement pour la Navigation Autonome

2.5.1 Fondements de l'apprentissage par renforcement

L'apprentissage par renforcement (AR) est un paradigme dans lequel un agent apprend à prendre des décisions par interaction avec un environnement. Formalisé comme un Processus de Décision de Markov (MDP), il est défini par le quadruplet $\mathcal{S}, \mathcal{A}, \mathcal{R}, \gamma$ où :

- $\mathcal{S}$ est l'espace d'états (observations du robot)
- $\mathcal{A}$ est l'espace d'actions (commandes de vitesse)
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ est la fonction de récompense
- $\gamma \in [0, 1]$ est le facteur d'actualisation temporelle

À chaque pas de temps t , l'agent observe l'état s_t , choisit une action a_t selon sa politique $\pi(a_t|s_t)$, reçoit une récompense r_t et transite vers un nouvel état s_{t+1} . L'objectif est de maximiser le rendement espéré $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$.

2.5.2 Apprentissage par renforcement profond

L'apprentissage par renforcement profond (DRL) combine les méthodes AR avec les réseaux de neurones pour approximer les fonctions de valeur ou les politiques dans des espaces d'états de haute dimension. Les algorithmes majeurs incluent :

- **DQN (Deep Q-Network)** : Approxime la fonction $Q(s, a)$ par un réseau de neurones. Limité aux espaces d'actions discrets, instable dans les espaces de grande dimension.
- **A3C (Asynchronous Advantage Actor-Critic)** : Apprentissage parallèle asynchrone avec architecture acteur-critique. Stable mais complexe à déboguer.
- **PPO (Proximal Policy Optimization) [9]** : Algorithme acteur-critique on-policy avec contrainte de mise à jour proximale (clipping du ratio $r_t \theta = \pi_\theta(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$ dans l'intervalle $[1 - \epsilon, 1 + \epsilon]$, avec $\epsilon = 0.2$). Supporte nativement les espaces d'actions continus (v, ω) , offre une grande stabilité d'entraînement et une implémentation accessible via Stable-Baselines3. C'est l'algorithme retenu dans ce travail.

2.5.3 Applications en robotique

L'AR est utilisé en robotique pour l'évitement d'obstacles, la planification de chemin et la navigation autonome. Son principal avantage réside dans sa capacité à apprendre des comportements adaptatifs sans modèle explicite de l'environnement, particulièrement précieux dans les scènes dynamiques où les obstacles changent de position de manière imprévisible. Cependant, les défis restent nombreux : temps d'entraînement longs, instabilité potentielle de l'apprentissage, et difficulté de transfert de la simulation au monde réel (*sim-to-real gap*). Dans ce travail, l'entraînement dans Gazebo avec le Turtlebot3 et l'utilisation de la perception stéréo comme entrée de l'agent visent à réduire ce fossé en rendant les observations du simulateur plus proches des observations réelles.

2.6 ROS et Gazebo pour le Développement Robotique

2.6.1 Robot Operating System (ROS)

ROS [10] est un framework middleware open-source qui fournit une infrastructure de communication décentralisée pour les systèmes robotiques. Son architecture en nœuds (*nodes*) communicants via des topics (publications/ abonnements) et des services (requête/réponse) permet un développement modulaire et réutilisable. Dans ce travail, les nœuds ROS assurent les fonctions suivantes : `psmnet_node` (inférence stéréo), `obstacle_detector` (RANSAC + DBSCAN), `rl_agent_node` (politique PPO), et `reward_calculator` (évaluation des récompenses). La synchronisation temporelle entre le nœud PSMNet (10 Hz) et l’agent PPO (10 Hz) est assurée par un `ApproximateTimeSynchronizer`.

2.6.2 Environnement de simulation Gazebo

Gazebo est un simulateur physique réaliste qui modélise les robots et leurs environnements avec une physique de corps rigides, des capteurs simulés (caméras, LiDAR, IMU) et des actionneurs. Il est entièrement intégré à ROS et permet de tester les algorithmes de navigation en toute sécurité avant tout déploiement sur un robot réel. Dans ce travail, Gazebo héberge le Turtlebot3 Waffle équipé d’une paire stéréo frontale simulée (baseline 0,12 m, résolution 640×480 px) et trois mondes personnalisés servant aux scénarios d’évaluation.

2.7 Analyse Comparative

2.7.1 Approches de mise en correspondance stéréo

TABLE 2.1 – Comparaison quantitative des méthodes de mise en correspondance stéréo (benchmark KITTI 2015)

Méthode	Type	EPE (px) ↓	D1-all (%) ↓	Latence	Fine-tune
BM (OpenCV)	Classique	4,2	8,1	~20 ms	N/A
SGM	Classique	3,5	6,5	~50 ms	N/A
DispNet	CNN	1,8	4,3	~60 ms	Moyen
PSMNet	CNN 3D	1,1	2,3	~180 ms	Facile
CREStereo	CNN + Att.	0,9	1,9	~200 ms	Moyen
RAFT-Stereo	CNN + GRU	0,7	1,6	~180 ms	Complexe

Le choix de PSMNet repose sur son meilleur compromis entre précision (EPE 1,1 px) et facilité de fine-tuning sur TartanGround dans le cadre d’un mémoire de master, en particulier grâce à la disponibilité de poids pré-entraînés sur SceneFlow et d’un code officiel bien documenté.

2.7.2 Approches de navigation

Le Tableau 2.2 compare les principales stratégies de navigation pour robots mobiles, incluant les approches classiques, par apprentissage par renforcement et hybrides.

TABLE 2.2 – Comparaison des stratégies de navigation pour robots mobiles

Approche	Adaptabilité	Complexité	Données requises
Classique (A*, DWA)	Faible	Faible	Carte a priori
Q-Learning tabulaire	Faible	Faible	Entraînement discret
DQN	Moyenne	Moyenne	Entraînement (actions discrètes)
RL (PPO, SAC)	Élevée	Moyenne	Entraînement Gazebo
Hybride	Intermédiaire	Élevée	Carte + entraînement

Approches classiques (A*, DWA) Ces planificateurs reposent sur une carte d’occupation explicite : A* assure l’optimalité du chemin global, DWA gère l’évitement local en échantillonnant l’espace cinématique admissible. Déterministes et bien supportés par Nav2 (ROS 2), ils se dégradent cependant face aux obstacles dynamiques et restent incapables de s’adapter à des configurations non cartographiées. Ils constituent la baseline de référence dans ce travail.

Q-Learning et DQN Le Q-Learning tabulaire apprend une table de valeurs $Q_{s,a}$ par essais et erreurs, sans modèle de l’environnement. Son applicabilité se limite toutefois aux espaces d’états et d’actions discrets et de faible dimension, ce qui le rend inadapté à la navigation en espace continu. DQN étend ce principe aux espaces d’états de grande dimension en approximant $Q_{s,a}$ par un réseau de neurones, mais reste contraint aux actions discrètes, incompatibles avec les commandes continues v, ω du Turtlebot3. Ces deux approches sont donc écartées pour ce travail.

Apprentissage par renforcement continu (PPO, SAC) PPO et SAC opèrent nativement sur des espaces d’actions continus. PPO, retenu dans ce travail, offre une stabilité d’entraînement supérieure grâce à son mécanisme de *clipping* du ratio de politique, et s’intègre directement via Stable-Baselines3 [9]. Sa capacité à exploiter des observations hétérogènes — carte de profondeur, orientation vers l’objectif, vitesse courante — en fait l’approche la mieux adaptée aux contraintes du Turtlebot3 dans des environnements dynamiques simulés sous Gazebo.

Approches hybrides Ces méthodes combinent un planificateur global géométrique et un agent RL pour l’évitement local. Bien qu’elles offrent une adaptabilité intermédiaire, leur complexité d’intégration est sensiblement plus élevée. Elles représentent une perspective d’évolution naturelle du système proposé, mais ne sont pas retenues dans le cadre de ce mémoire.

Le choix de PPO se justifie donc par sa compatibilité avec les actions continues du robot $v \in [0, 0.5]$ m/s, $\omega \in [-1, 1]$ rad/s, sa stabilité d’entraînement et la disponibilité d’une implémentation de référence, avantages que ne partagent ni le Q-Learning tabulaire ni DQN.

2.8 Discussion

La littérature montre que l'apprentissage profond améliore significativement la précision de la perception, en particulier pour la vision stéréo et la détection d'objets. Cependant, cela s'accompagne d'exigences computationnelles accrues. La comparaison des benchmarks (Tableau 2.1) révèle une réduction de l'EPE de l'ordre de 67 % entre SGM (3,5 px) et PSMNet (1,1 px), soit un gain considérable pour la sécurité de navigation.

Pour les systèmes robotiques temps réel, un compromis doit être trouvé entre précision, vitesse et complexité du système. Dans ce travail, ce compromis est adressé par : (1) le choix de PSMNet comme module de perception (précision élevée à complexité maîtrisée), (2) l'export ONNX du modèle pour réduire la latence en dessous de 100 ms, et (3) l'utilisation de PPO (stabilité éprouvée, actions continues) pour la navigation.

2.9 Conclusion

Ce chapitre a présenté les principaux concepts théoriques relatifs à la vision stéréo, l'apprentissage profond, l'apprentissage par renforcement et les frameworks robotiques. L'analyse comparative a permis de justifier les choix technologiques du présent travail :

- PSMNet est l'architecture de perception stéréo retenue, pour son précision et sa facilité de fine-tuning sur TartanGround.
- PPO est l'algorithme de navigation retenu, pour sa stabilité sur les actions continues et son implémentation via Stable-Baselines3.
- TartanGround est le jeu de données retenu, pour son adéquation aux robots de sol différentiels en environnements intérieurs.
- ROS/Gazebo est le framework de développement et simulation retenu, pour son intégration native et sa richesse d'outils.

Ces éléments constituent la fondation du système proposé dans ce travail, dont la description détaillée fait l'objet des chapitres suivants.

Chapitre 3

Module de Perception Stéréoscopique

3.1 Introduction

Le module de perception stéréoscopique constitue le composant technique central du système de navigation autonome proposé. Son objectif principal est de permettre au robot de percevoir son environnement en trois dimensions à partir d'une paire de caméras stéréo, en exploitant des techniques d'apprentissage profond pour dépasser les limitations des approches classiques de mise en correspondance stéréo.

Ce module combine le traitement d'images stéréo et un réseau de neurones convolutifs (PSMNet) pour générer des cartes de disparité denses en temps réel. Ces cartes sont ensuite converties en nuages de points 3D utilisés pour la segmentation des zones navigables et la délimitation des obstacles. L'intégration de ces composantes au sein de nœuds ROS permet une communication fluide avec l'agent de navigation PPO, garantissant une réactivité de l'ensemble du pipeline inférieure à 100 ms.

La Figure ?? illustre le flux de données du pipeline de perception, depuis l'acquisition des paires d'images stéréo jusqu'à la publication des marqueurs d'obstacles sur les topics ROS.

3.2 Description du Jeu de Données

3.2.1 TartanGround

Le développement et l'évaluation du modèle de perception stéréo reposent sur le jeu de données TartanGround [3], une ressource de référence pour la perception des robots de sol, publiée en 2025 par le Carnegie Mellon University AirLab. TartanGround a été spécifiquement conçu pour combler le vide laissé par les jeux de données automobiles (KITTI, nuScenes) vis-à-vis des robots de sol, dont le point de vue bas et les environnements de traversée diffèrent fondamentalement de ceux des véhicules.

Le jeu de données présente les caractéristiques suivantes :

- **Volume** : Plus de 1,44 million d'échantillons répartis sur 878 trajectoires dans 63 environnements photoréalistes (créés sous Unreal Engine 4).
- **Modalités** : Images RGB stéréo gauche/droite (640×640 px, champ de vue 90°), cartes de profondeur de vérité terrain (.npy, float32), segmentation sémantique, données LiDAR 32 faisceaux simulés, poses 6-DOF, données IMU.

- **Robots** : Trois types de plateformes : omnidirectionnel (omni), différentiel (diff) et quadrupède (anymal).
- **Environnements** : Catégorisés en 6 types : Intérieur, Nature, Rural, Urbain, Industriel, Historique/Thématique.
- **Licence** : Creative Commons Attribution 4.0 (CC BY 4.0).

Pour ce travail, le sous-ensemble **intérieur, robot différentiel** est utilisé en priorité, car il correspond le mieux aux conditions de déploiement du Turtlebot3 :

```

1 import tartanair as ta
2
3 ta.init('./data/tartanground')
4
5 INDOOR_ENVS = [
6     "AbandonedFactory2",    # Infra      Intérieur      Petit
7     "AmericanDiner",        # Domes     Intérieur      Moyen
8     "ArchVizTinyHouseDay",   # Domes     Intérieur      Petit
9     "CarWelding",            # Infra      Intérieur      Moyen +
10     "CoalMine",              # Infra      Intérieur      Moyen
11     "CountryHouse",          # Domes     Intérieur      Petit
12     "CyberPunkDowntown",     # Theme     Intérieur      Moyen +
13     "Dynamic"
14 ]
15 ta.download_ground(
16     env            = INDOOR_ENVS,
17     version        = ['diff'],          # robot différentiel =
18     modality        = ['image', 'depth', 'meta'], # uniquement le
19     camera_name     = ['lcam_front', 'rcam_front'], # paire stereo
20     unzip           = True
21 )

```

Listing 3.1 – Téléchargement du sous-ensemble pertinent via tartanairpy

3.2.2 Prétraitement des données

Avant l'entraînement, le jeu de données subit plusieurs étapes de prétraitement standardisées :

- **Redimensionnement** : Les images de 640×640 px sont redimensionnées à 256×512 px (rapport hauteur/largeur 1 :2) pour réduire l'empreinte mémoire GPU tout en conservant suffisamment de résolution pour les petits obstacles.

- **Normalisation** : Les valeurs de pixels sont normalisées selon les statistiques ImageNet (moyenne $\mu = 0,485, 0,456, 0,406$, écart-type $\sigma = 0,229, 0,224, 0,225$), ce qui facilite le fine-tuning à partir de poids pré-entraînés.
- **Augmentation de données** : En entraînement uniquement : recadrage aléatoire (crop à 90 % de la résolution), jitter chromatique (luminosité $\pm 20\%$, contraste $\pm 20\%$, saturation $\pm 20\%$). Le retournement horizontal est **interdit** pour une paire stéréo car il briserait la cohérence géométrique (l'image gauche deviendrait droite et vice versa).
- **Validation des disparités** : Les valeurs de disparité GT hors de la plage 0, 192 pixels sont masquées lors du calcul de la loss, pour éviter de superviser des pixels dont la correspondance n'est pas fiable (zones d'occultation, bords d'image).
- **Division entraînement/validation/test** : Les trajectoires sont réparties aléatoirement selon un ratio 85 %/10 %/5 %.

3.3 Modèle CNN de Mise en Correspondance Stéréo

3.3.1 Sélection du modèle

Parmi les architectures disponibles pour la mise en correspondance stéréo par apprentissage profond, PSMNet est retenu dans ce travail pour les raisons suivantes : (1) ses poids pré-entraînés sur SceneFlow (35 000 paires synthétiques) permettent un fine-tuning rapide ; (2) son code officiel est maintenu sur GitHub avec une documentation de qualité ; (3) ses performances ($EPE \approx 1,1$ px) sont suffisantes pour la navigation sécurisée du Turtlebot3 dans la plage 0,5–10 m ; (4) sa latence d'inférence (~ 180 ms), réduite à < 100 ms après export ONNX, est compatible avec la boucle ROS à 10 Hz.

Le pipeline général du modèle suit quatre étapes :

1. Extraction de caractéristiques à partir des images gauche et droite
2. Construction d'un volume de coût 4D par concaténation des caractéristiques
3. Régression de disparité par convolutions 3D et softargmax
4. Raffinement de la carte de disparité par supervision multi-échelle

3.3.2 Architecture du réseau

L'architecture PSMNet repose sur les composants suivants :

- **Réseau partagé avec module SPP** : Un extracteur de caractéristiques CNN commun (mêmes poids) est appliqué aux deux images stéréo, produisant des représentations de dimensions $B \times 128 \times H/8 \times W/8$. Le module SPP intègre des branches de pooling adaptatif à 4 résolutions (8, 16, 32, 64) pour capturer le contexte à différentes échelles spatiales, crucial pour la correspondance dans les zones à faible texture.
- **Volume de coût 4D** : Pour chaque niveau de disparité $d \in 0, D_{\max}/8$, les caractéristiques gauches et droites décalées sont concaténées : $\text{cost}[:, :, C, d, :, d] = f_L$ et $\text{cost}[:, :, C, :, d, :, d] =$

- **Agrégation par convolutions 3D (Hourglass) :** Deux modules hourglass 3D (encodeur-décodeur 3D avec connexions de saut) régularisent le volume de coût spatialement, supprimant le bruit et renforçant la cohérence des disparités estimées.
- **Régression par softargmax :** La disparité finale est obtenue par espérance sur la distribution de probabilité softmax du volume de coût : $\hat{d} = \frac{D_{\max}}{d=0} d \cdot \sigma - c_d$, garantissant une précision sub-pixel et la différentiabilité pour l'entraînement end-to-end.

3.4.1 Procédure d'entraînement

1. **Pré-entraînement sur SceneFlow** : Les poids officiels de PSMNet pré-entraînés sur le jeu de données SceneFlow (35 000 paires synthétiques, disponibles sur le repository officiel) sont utilisés comme point de départ. Ce pré-entraînement fournit un point de départ riche en représentations de bas niveau (bords, textures) transférables à TartanGround.
2. **Fine-tuning sur TartanGround (intérieur, diff)** : Le modèle est ensuite affiné sur notre sous-ensemble pendant 300 époques avec :
 - **Fonction de perte** : Smooth L1 avec supervision multi-échelle : $\mathcal{L} = 0,5\mathcal{L}_1 + 0,7\mathcal{L}_2 + 1,0\mathcal{L}_3$, où chaque $\mathcal{L}_i$ est calculée uniquement sur les pixels valides (masque $d > 0$ et $d < 192$ px).
 - **Optimiseur** : Adam, taux d'apprentissage initial 10^{-3} , réduit d'un facteur 10 à l'époque 200.
 - **Taille de batch** : 4 (contrainte GPU 16 Go).
 - **Durée** : $\approx 3\text{--}4$ h sur GPU NVIDIA A100 (Colab Pro).

Pour évaluer les performances du modèle, plusieurs métriques standardisées sont utilisées :

- **EPE (*End-Point Error*)** : Erreur absolue moyenne en pixels entre la disparité prédite et la vérité terrain : $EPE = \frac{1}{|\mathcal{M}|} \sum_{p \in \mathcal{M}} |\hat{d}_p - d_p^*|$, calculée uniquement sur les pixels valides. C'est la métrique principale de ce travail.
- **D1-all** : Pourcentage de pixels avec une erreur supérieure à 3 px ET supérieure à 5 % de la vérité terrain : $D1 = \frac{1}{|\mathcal{M}|} \sum_p \mathbf{1}(|\hat{d}_p - d_p^*| > 3 \wedge \frac{|\hat{d}_p - d_p^*|}{d_p^*} > 0,05)$.
- **Bad2.0 et Bad3.0** : Pourcentage de pixels avec une erreur respectivement supérieure à 2 px et 3 px.

Les résultats attendus après fine-tuning sur TartanGround sont : $EPE \approx 1,0\text{--}1,5$ px, $D1\text{-all} \approx 2,5\text{--}3,5$ %, représentant une amélioration de l'ordre de 66 % par rapport à la baseline SGM ($EPE \approx 3,5$ px).

3.5 Génération de Cartes de Disparité

Une fois entraîné, le modèle génère des cartes de disparité denses à partir des entrées stéréo. Ces cartes encodent la profondeur relative de chaque pixel de la scène. La carte de disparité $\hat{D}$ est ensuite :

1. **Convertie en carte de profondeur** : $Z = f \cdot B \hat{D}$, où f est la focale de la caméra (en pixels) et B la baseline stéréo (en mètres). Un filtre médian 5×5 est appliqué pour supprimer les pics de bruit isolés.
2. **Filtrée** : Les pixels avec $Z < 0,3$ m (trop proches, réflexions sur le châssis) ou $Z > 10$ m (précision insuffisante) sont marqués comme invalides.
3. **Transformée en nuage de points 3D** : Par rétroprojection : $X = u - c_x \cdot Z f_x$, $Y = v - c_y \cdot Z f_y$, ce qui permet de positionner chaque pixel valide dans l'espace 3D de la caméra.

Cette étape est critique pour transformer les données visuelles en informations spatiales exploitables par le système de navigation.

3.6 Détection d'Obstacles

3.6.1 Détection par analyse de profondeur

La détection d'obstacles repose sur l'analyse du nuage de points 3D issu de la carte de profondeur. Le pipeline comprend deux étapes complémentaires :

Segmentation du plan du sol (RANSAC). L'algorithme RANSAC (*Random Sample Consensus*) est appliqué au nuage de points pour estimer les paramètres du plan du sol ($ax \ by \ cz \ d = 0$). En échantillonnant itérativement des triplets de points minimaux et en mesurant le nombre de points intra-seuil (*inliers*, seuil = 0,02 m), il identifie robustement le plan dominant (le sol) même en présence de nombreux points aberrants. Les points classifiés comme *inliers* sont retirés du nuage, laissant uniquement les points obstacles.

Regroupement des obstacles (DBSCAN). L'algorithme DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) est ensuite appliqué au nuage de points non-sol pour regrouper les points spatialement proches en clusters distincts. Avec les paramètres $\varepsilon = 0,3$ m et $\text{min_points} = 10$, chaque cluster correspond à un obstacle individuel. Pour chaque cluster, une boîte englobante 3D est calculée et publiée sur le topic ROS `/obstacles/markers`.

3.6.2 Détection assistée par CNN

Pour améliorer la robustesse de la détection, la carte de profondeur 64×64 (version compressée) est directement intégrée dans le vecteur d'état de l'agent PPO, permettant au réseau de politique d'apprendre implicitement à éviter les obstacles sans détection explicite. Cette approche *end-to-end*

complète la détection géométrique par RANSAC+DBSCAN en offrant une réactivité aux obstacles de faible relief difficilement segmentables par la seule géométrie.

3.7 Intégration dans le Pipeline de Navigation

La sortie du module de perception est intégrée dans le système de navigation du robot via les nœuds ROS. Le processus comprend :

1. **Publication de la carte de profondeur** : Le nœud `psmnet_node` publie la carte de profondeur sur `/stereo/depth` à 10 Hz.
2. **Mise à jour de la grille d'occupation** : Le nœud `obstacle_detector` convertit le nuage de points obstacles en grille d'occupation locale (6×6 m, résolution 5 cm/cellule) publiée sur `/obstacles/zones`.
3. **Fourniture d'informations à l'agent PPO** : Le nœud `rl_agent_node` souscrit aux topics `/stereo/depth`, `/odom` et `/goal` pour construire son vecteur d'état et calculer la commande de vitesse (`/cmd_vel`).

Cette intégration garantit que le robot peut réagir aux obstacles dynamiques en temps réel, avec une latence totale du pipeline (acquisition → commande) inférieure à 200 ms.

3.8 Discussion

Le module de perception stéréo basé sur CNN offre plusieurs avantages décisifs pour la navigation autonome du Turtlebot3 :

- **Estimation de profondeur précise** : PSMNet atteint $EPE \approx 1,1$ px sur KITTI et devrait atteindre des performances similaires sur TartanGround après fine-tuning, soit une précision nettement supérieure à SGM ($EPE \approx 3,5$ px). Cette précision est suffisante pour distinguer des obstacles aussi fins qu'une chaise de bureau ou un câble au sol.
- **Détection robuste des obstacles** : La combinaison RANSAC (sol) + DBSCAN (clusters) produit des boîtes englobantes 3D en ~ 30 ms, compatible avec la boucle à 10 Hz. La robustesse aux surfaces sans texture (murs peints), problème des méthodes classiques, est assurée par les représentations apprises de PSMNet.
- **Sécurité de navigation améliorée** : L'alerte de collision (seuil 0,5 m) et la carte d'occupation locale permettent à l'agent PPO d'anticiper les obstacles avant contact, réduisant le taux de collision attendu en dessous de 5 %.

Les défis identifiés incluent : (1) les exigences computationnelles élevées de PSMNet (~ 180 ms, adressées par l'export ONNX); (2) la dépendance à la qualité du jeu de données (TartanGround répond partiellement à ce défi par sa richesse et sa diversité); (3) la sensibilité aux décalages de domaine entre la simulation et le robot réel (*sim-to-real gap*), qui reste un axe de travail futur.

3.9 Conclusion

Ce chapitre a présenté la conception et l'implémentation du module de perception stéréoscopique basé sur CNN. En combinant PSMNet, entraîné sur le sous-ensemble intérieur de TartanGround, avec une chaîne de traitement RANSAC+DBSCAN intégrée dans ROS, le système atteint une estimation de profondeur fiable et une détection d'obstacles en temps réel adaptées aux contraintes du Turtlebot3. Ce module constitue la fondation du système de navigation autonome développé dans ce travail et joue un rôle déterminant dans l'opération sûre et efficace du robot.

Chapitre 4

Apprentissage par Renforcement pour la Navigation

4.1 Introduction

Le module de perception stéréoscopique décrit au chapitre précédent fournit une représentation tridimensionnelle de l’environnement du robot sous forme de cartes de profondeur et de marqueurs d’obstacles. Cette représentation constitue l’entrée d’un agent de navigation dont la mission est de planifier des trajectoires sans collision vers un objectif donné, dans des environnements dynamiques et variables. Ce chapitre détaille la conception et l’entraînement de cet agent, fondé sur l’algorithme d’apprentissage par renforcement proximal PPO (Proximal Policy Optimization).

La navigation autonome est formalisée comme un Processus de Décision de Markov (MDP), cadre théorique qui permet de modéliser les interactions séquentielles entre le robot et son environnement. L’agent apprend une politique de décision optimale par interaction répétée avec la simulation Gazebo, sans connaissance préalable de la géométrie de l’environnement. Une stratégie de curriculum learning permet d’accélérer la convergence en exposant progressivement l’agent à des scénarios de complexité croissante.

4.2 Formulation du Processus de Décision de Markov

4.2.1 Espace d’états

Le vecteur d’état est un dictionnaire hétérogène traité par une architecture `MultiInputPolicy` de `Stable-Baselines3`. Il est composé de trois sources d’observation complémentaires, décrites dans le Tableau 4.1.

TABLE 4.1 – Composantes du vecteur d’état de l’agent PPO

Composante	Dimension	Source ROS 2	Description
depth	64×64	/stereo/depth	Carte de profondeur compressée, $Z \in [0, 10 \text{ m}]$
goal	3	/goal_pose + /odom	$\Delta x, \Delta y, \Delta \theta$ vers l’objectif
vel	2	/odom	Vitesse courante v, ω

La carte de profondeur 64×64 est obtenue par redimensionnement bilinéaire de la sortie PSMNet ($256 \times 512 \text{ px}$). Elle encode implicitement la géométrie des obstacles sans nécessiter de

détection géométrique explicite à ce stade, permettant à l'agent d'apprendre un comportement d'évitement en boucle fermée. Le vecteur d'orientation vers l'objectif Δx , Δy , $\Delta \theta$ est exprimé dans le repère du robot et mis à jour à chaque pas de temps depuis les topics `/odom` et `/goal_pose`. La vitesse courante v , ω informe l'agent de son état cinématique, nécessaire pour anticiper les effets inertiel de ses commandes.

L'espace d'observations est formalisé par :

$$\mathcal{S} = \mathbb{R}^{64 \times 64} \times -10, 10^3 \times -2, 2^2 \quad (4.1)$$

4.2.2 Espace d'actions

L'espace d'actions est **continu** et bidimensionnel, adapté à la cinématique différentielle du Turtlebot3 Waffle :

$$\mathcal{A} = \{v, \omega \mid v \in 0, 0,5 \text{ m/s}, \omega \in -1, 1 \text{ rad/s}\} \quad (4.2)$$

Le choix d'un espace d'actions continu est déterminant : les approches discrètes (DQN, Q-Learning tabulaire) imposent une quantification des vitesses qui génère des trajectoires saccadées, incompatibles avec la dynamique du Turtlebot3 et sources d'instabilité mécanique. PPO supporte nativement cet espace via une politique gaussienne paramétrique $\pi_{\theta}a|s = \mathcal{N}_{\mu_{\theta}s, \sigma_{\theta}^2}$.

4.2.3 Fonction de récompense

La fonction de récompense r_t est conçue pour encourager la progression vers l'objectif tout en pénalisant les comportements dangereux :

$$r_t = \begin{cases} 100 & \text{si } \|\mathbf{p}_t - \mathbf{g}\| < 0,3 \text{ m (objectif atteint)} \\ -100 & \text{si collision détectée (topic /collision)} \\ 2 \cdot \Delta d_t & \text{récompense de progression} \\ -\frac{5}{d_{\min,t}} & \text{si } d_{\min,t} < 1,0 \text{ m (zone de danger)} \end{cases} \quad (4.3)$$

où $\Delta d_t = \|\mathbf{p}_{t-1} - \mathbf{g}\| - \|\mathbf{p}_t - \mathbf{g}\|$ est la réduction de distance à l'objectif au pas t , et $d_{\min,t}$ est la distance minimale aux obstacles publiée par le nœud `obstacle_detector`. Ce terme répulsif en $1/d$ crée un gradient continu qui pousse l'agent à maintenir une marge de sécurité sans déclencher la pénalité de collision.

4.2.4 Facteur d'actualisation et horizon

Le facteur d'actualisation $\gamma = 0,99$ favorise la prise en compte des récompenses à long terme, ce qui est essentiel pour un robot devant anticiper les obstacles à plusieurs pas de décision. L'horizon d'un épisode est limité à $T = 500$ pas (soit 50 s à 10 Hz) ; au-delà, l'épisode est déclaré *timeout*.

4.3 Architecture de l'Agent PPO

4.3.1 Choix algorithmique

Parmi les algorithmes de RL profond applicables à la navigation en espace continu, PPO [9] est retenu pour les raisons suivantes :

- **Stabilité d'entraînement** : le mécanisme de *clipping* du ratio de politique $r_t \theta = \frac{\pi_{\theta} a | s}{\pi_{\theta_{\text{old}}} a | s}$ dans l'intervalle $1 - \varepsilon, 1 + \varepsilon$ (avec $\varepsilon = 0,2$) prévient les mises à jour trop agressives qui destabilisent SAC ou A3C.
- **Actions continues** : contrairement à DQN et Q-Learning, PPO supporte nativement l'espace $\mathcal{A}$ continu du Turtlebot3.
- **Implémentation accessible** : Stable-Baselines3 fournit une implémentation de référence avec `MultiInputPolicy`, directement compatible avec les espaces d'observation dictionnaires de l'environnement Gymnasium.

4.3.2 Hyperparamètres

Le Tableau 4.2 regroupe les hyperparamètres retenus après une phase d'exploration manuelle.

TABLE 4.2 – Hyperparamètres de l'agent PPO

Hyperparamètre	Valeur	Justification
Taux d'apprentissage α	3×10^{-4}	Standard Adam pour RL continu
Pas de collecte n_steps	2048	Compromis variance/biais
Taille de batch	64	Contrainte mémoire GPU
Facteur d'actualisation γ	0,99	Horizon long (navigation)
Clipping ε	0,2	Valeur PPO standard
Coefficient entropie	0,01	Exploration suffisante
Epochs par mise à jour	10	Réutilisation des transitions
Total de pas d'entraînement	500 000	Convergence observée

4.3.3 Architecture du réseau de politique

La politique PPO repose sur un réseau MLP partagé entre l'acteur et le critique, précédé d'un encodeur spécifique pour chaque modalité d'entrée :

1. **Encodeur profondeur** : couche de convolution 8×8 (stride 4) suivie d'une couche dense 256 — réduit la carte 64×64 en un vecteur de 256 dimensions.
2. **Encodeur scalaire** : couche dense 64 appliquée à la concaténation $\Delta x, \Delta y, \Delta \theta, v, \omega$.
3. **Tronc partagé** : deux couches denses 256-256 avec activation ReLU, communes à l'acteur et au critique.
4. **Tête acteur** : couche dense $|\mathcal{A}| = 2$ produisant μ_v, μ_ω , les moyennes de la politique gaussienne.
5. **Tête critique** : couche dense 1 produisant l'estimation de valeur V_s .

4.4 Stratégie de Curriculum Learning

Un entraînement direct sur les scénarios les plus complexes (obstacles mobiles, entrepôt) converge lentement et produit des politiques sous-optimales par manque de signal de récompense positif initial. Le curriculum learning résout ce problème en exposant progressivement l’agent à des environnements de difficulté croissante, selon les trois phases du Tableau 4.3.

TABLE 4.3 – Phases du curriculum learning

Phase	Environnement	Difficulté	Pas d’entraînement
1	Obstacles statiques uniquement	Faible	0–200 000
2	Obstacles mobiles lents (0,2 m/s)	Moyenne	200 000–350 000
3	Obstacles mobiles rapides (1,0 m/s)	Élevée	350 000–500 000

La transition entre phases est déclenchée automatiquement lorsque le taux de succès moyen sur les 100 derniers épisodes dépasse 80 %, ce qui garantit que l’agent maîtrise le niveau courant avant d’être exposé au suivant. Le modèle sauvegardé à la fin de chaque phase (`ppo_phase{1,2,3}.zip`) permet de reprendre l’entraînement sans repartir de zéro en cas d’instabilité.

4.5 Environnement d’Entraînement sous ROS 2 / Gazebo

L’environnement d’apprentissage est implémenté comme une classe `TurtlebotStereoEnv` héritant de `gymnasium.Env`, connectée en temps réel à la simulation Gazebo via des topics ROS 2. Les trois mondes Gazebo utilisés pendant l’entraînement sont :

- **Corridor** (`corridor.world`) : couloir de $10 \times 2,5$ m, obstacles statiques en quinconce — utilisé en phase 1.
- **Espace ouvert** (`open_space.world`) : espace de 8×8 m avec obstacles statiques et un acteur dynamique à 0,5 m/s — phases 2 et 3.
- **Entrepôt** (`warehouse.world`) : espace de 12×8 m avec étagères (surfaces sans texture, difficiles pour la stéréo) et obstacles mixtes — phase 3.

À chaque appel à `reset()`, la simulation Gazebo est réinitialisée via le topic `/reset_simulation`, la position du robot est replacée à l’origine, et un nouvel objectif est tiré aléatoirement dans l’espace navigable.

4.6 Protocole d’Évaluation sur 50 Trajectoires

4.6.1 Conception du protocole

L’évaluation du système complet repose sur 50 trajectoires de test réparties sur les trois scénarios selon le Tableau 4.4.

Les graines aléatoires (*seeds*) sont fixées pour assurer la reproductibilité des expériences. Pour chaque trajectoire, le robot part d’une position initiale fixée et doit atteindre un objectif à une distance euclidienne d’environ 4 m en moins de 500 pas de décision (soit 50 s à 10 Hz).

TABLE 4.4 – Répartition des trajectoires de test par scénario

Scénario	Trajectoires	Caractéristiques	Seeds
A — Couloir	15	Obstacles statiques, espace contraint	0–14
B — Espace ouvert	20	10 statiques + 10 avec obstacles mobiles	15–34
C — Entrepôt	15	Géométrie complexe, obstacles mixtes	35–49
Total	50		

4.6.2 Métriques d’évaluation

Une trajectoire est classée selon trois issues mutuellement exclusives :

- **Succès** : objectif atteint ($d < 0,3$ m) sans collision.
- **Échec par collision** : contact avec un obstacle détecté par le topic `/collision`.
- **Timeout** : limite de 500 pas dépassée sans atteindre l’objectif.

Les métriques quantitatives calculées sur l’ensemble des 50 trajectoires sont définies dans le Tableau 4.5.

TABLE 4.5 – Métriques d’évaluation du système de navigation autonome

Métrique	Définition	Cible
Taux de succès (%)	$\frac{\text{trajectoires réussies}}{50} \times 100$	$> 85 \%$
Taux de collision (%)	$\frac{\text{trajectoires avec collision}}{50} \times 100$	$< 5 \%$
Taux de timeout (%)	$\frac{\text{trajectoires timeout}}{50} \times 100$	$< 10 \%$
Longueur normalisée	$L_{\text{robot}} / L_{\text{optimal}}$	$< 1,5$
Récompense cumulée G_t	$\sum_{k=0}^T \gamma^k r_{tk}$ par épisode	Convergence stable
Latence pipeline (ms)	Délai acquisition $\rightarrow$ <code>/cmd_vel</code>	< 200 ms

4.6.3 Comparaison avec la baseline classique

Chaque métrique est comparée à une baseline SGM + planificateur A* classique :

- **Perception baseline** : algorithme SGM d’OpenCV (StereoSGBM, paramètres optimisés) en remplacement de PSMNet.
- **Navigation baseline** : planificateur A* global avec évitement local DWA (Dynamic Window Approach), configuré via Nav2 sous ROS 2 Humble.

Cette comparaison permet de quantifier séparément l’apport de l’apprentissage profond en perception (PSMNet vs SGM) et en navigation (PPO vs A*+DWA) sur les performances globales du système.

4.7 Conclusion

Ce chapitre a présenté la conception complète de l’agent de navigation par renforcement : la formulation MDP avec son espace d’états hétérogène issu de la perception stéréo, l’espace d’actions continu adapté à la cinématique du Turtlebot3, et la fonction de récompense combinant progression vers l’objectif et répulsion des obstacles. L’architecture PPO avec `MultiInputPolicy`

et le curriculum learning en trois phases constituent les deux leviers principaux de la robustesse et de la convergence de l'agent. Le protocole d'évaluation sur 50 trajectoires réparties sur trois scénarios de complexité croissante fournit un cadre expérimental rigoureux et reproductible pour la comparaison avec les baselines classiques, dont les résultats sont rapportés au Chapitre 7.

Chapitre 5

Intégration du Système et Configuration Expérimentale

5.1 Introduction

L'intégration du système constitue l'étape de synthèse du présent travail : elle réunit le module de perception stéréoscopique (PSMNet, détection RANSAC+DBSCAN) et l'agent de navigation par renforcement (PPO) au sein d'une architecture logicielle unifiée, déployée sur la plateforme de simulation ROS 2 Humble / Gazebo 11 / RViz2 avec le robot Turtlebot3 Waffle. Ce chapitre décrit de manière exhaustive l'environnement matériel et logiciel, la configuration de la simulation, l'architecture des nœuds ROS 2, les scénarios de test conçus pour l'évaluation et la méthodologie d'évaluation quantitative adoptée.

Contrairement à ROS 1 Noetic, qui repose sur un nœud maître centralisé (*rosmaster*), ROS 2 Humble adopte un modèle de communication distribué fondé sur le protocole DDS (*Data Distribution Service*), ce qui améliore la robustesse, la scalabilité et la latence du système. Sur Ubuntu 22.04 LTS (*Jammy Jellyfish*), ROS 2 Humble Hawksbill est la distribution supportée officiellement jusqu'en mai 2027, en faisant le choix naturel pour un déploiement pérenne.

5.2 Environnement Matériel et Logiciel

5.2.1 Configuration matérielle

Les expériences sont conduites sur la configuration suivante :

TABLE 5.1 – Spécifications matérielles de la station de développement

Composant	Spécification
Système d'exploitation	Ubuntu 22.04.4 LTS (<i>Jammy Jellyfish</i>), noyau 6.5
Processeur	Intel Core i7-12700H (14 cœurs, 4,7 GHz Turbo)
RAM	16 Go DDR5-4800
GPU	NVIDIA RTX 3070 (8 Go GDDR6, CUDA 12.1)
Stockage	SSD NVMe 512 Go
Robot simulé	Turtlebot3 Waffle (modèle URDF officiel, <i>diff</i>)

5.2.2 Environnement logiciel

TABLE 5.2 – Spécifications logicielles de l’environnement d’intégration

Logiciel	Version
ROS 2 Humble Hawksbill	0.10.0 (support LTS mai 2027)
Gazebo Classic	11.14.0 (gazebo11)
RViz2	11.2.x
Python	3.10.12
PyTorch	2.1.2 (CUDA 12.1)
Stable-Baselines3	2.3.2
Open3D	0.18.0
OpenCV	4.8.0
colcon	0.3.23

5.2.3 Installation de l’environnement ROS 2 Humble

L’installation suit la procédure officielle ROBOTIS pour ROS 2 Humble sur Ubuntu 22.04 [10].

Les commandes ci-dessous résument les étapes essentielles :

```

1 # 1. Configurer les sources APT ROS 2
2 sudo apt install software-properties-common curl -y
3 sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/
   master/\
4 ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
5 echo "deb [arch=$(dpkg --print-architecture) \
6     signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
7     http://packages.ros.org/ros2/ubuntu jammy main" | \
8     sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
9
10 # 2. Installer ROS 2 Humble complet
11 sudo apt update && sudo apt upgrade -y
12 sudo apt install ros-humble-desktop-full -y
13
14 # 3. Installer Gazebo Classic 11 (gazebo11 distinct de Gazebo
   Ignition)
15 # IMPORTANT : utiliser gazebo11 (classic), pas gazebo-fortress/
   harmonic
16 sudo apt install gazebo11 ros-humble-gazebo-ros-pkgs \
17     ros-humble-gazebo-ros2-control -y
18
19 # 4. Installer les packages Turtlebot3
20 sudo apt install ros-humble-turtlebot3* \

```

```

21         ros-humble-nav2-bringup \
22         ros-humble-navigation2 \
23         ros-humble-cartographer \
24         ros-humble-cartographer-ros \
25         ros-humble-cv-bridge \
26         ros-humble-image-transport \
27         ros-humble-stereo-image-proc \
28         ros-humble-vision-opencv \
29         ros-humble-rviz2 -y
30
31 # 5. Variables d'environnement (a ajouter dans ~/.bashrc)
32 echo "source_/opt/ros/humble/setup.bash" >> ~/.bashrc
33 echo "export_TURTLEBOT3_MODEL=waffle" >> ~/.bashrc
34 echo "export_GAZEBO_MODEL_PATH=\$GAZEBO_MODEL_PATH:\
35 /opt/ros/humble/share/turtlebot3_gazebo/models" >> ~/.bashrc
36 source ~/.bashrc

```

Listing 5.1 – Installation de ROS 2 Humble et des dépendances Turtlebot3

```

1 # Cloner les sources Turtlebot3 (branche humble)
2 mkdir -p ~/stereo_nav_ws/src && cd ~/stereo_nav_ws/src
3 git clone -b humble \
4     https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
5 git clone -b humble \
6     https://github.com/ROBOTIS-GIT/turtlebot3.git
7 git clone -b humble \
8     https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
9 git clone -b humble \
10    https://github.com/ROBOTIS-GIT/DynamixelSDK.git
11
12 # Installer les packages Python de perception/navigation
13 pip install torch torchvision stable-baselines3 open3d \
14     opencv-python scikit-learn wandb pyyaml
15
16 # Construire le workspace
17 cd ~/stereo_nav_ws
18 rosdep install -i --from-path src --rosdistro humble -y
19 colcon build --symlink-install
20 echo "source_~/stereo_nav_ws/install/setup.bash" >> ~/.bashrc
21 source ~/.bashrc

```

Listing 5.2 – Construction du workspace colcon pour les packages personnalisés

Note importante sur Gazebo : Sous ROS 2 Humble, deux versions de Gazebo coexistent : Gazebo Classic 11 (gazebo11) et Gazebo Fortress/ Harmonic (*Ignition*). Dans ce travail, Gazebo Classic 11

est utilisé, car il offre une meilleure compatibilité avec les packages Turtlebot3 disponibles en branche humble et avec la bibliothèque gazebo_ros_pkgs. Certains packages Turtlebot3 présentent des conflits de dépendances avec Ignition Gazebo qui peuvent bloquer la compilation [?]; Gazebo 11 est exempt de ces problèmes.

5.3 Structure du Workspace ROS 2

5.3.1 Organisation des packages

Le workspace stereo_nav_ws est organisé selon les conventions ROS 2 :

```

1 stereo_nav_ws/
2 |-- src/
3 |   |-- stereo_vision/                # Package de perception stereo
4 |   |   |-- package.xml
5 |   |   |-- setup.py
6 |   |   |-- config/
7 |   |   |   '-- stereo_params.yaml
8 |   |   |-- launch/
9 |   |   |   '-- stereo_perception.launch.py
10 |   |   '-- stereo_vision/
11 |   |       |-- psmnet_node.py        # Inference PSMNet (noeud ROS
12 |   |           '-- obstacle_detector.py    # RANSAC + DBSCAN
13 |   |
14 |   |-- rl_navigation/                # Package de navigation PPO
15 |   |   |-- package.xml
16 |   |   |-- setup.py
17 |   |   |-- launch/
18 |   |   |   |-- train.launch.py        # Entraînement PPO
19 |   |   |   '-- deploy.launch.py        # Déploiement politique
20 |   |   '-- rl_navigation/
21 |   |       |-- rl_agent_node.py        # Agent PPO (noeud ROS 2)
22 |   |       |-- gym_env.py              # Environnement Gymnasium+ROS
23 |   |           |-- reward_node.py        # Calcul des récompenses
24 |   |           '-- train_ppo.py         # Script d'entraînement
25 |   |
26 |   |-- turtlebot3_stereo/            # Config robot + URDF stereo
27 |   |   |-- urdf/
28 |   |   |   '-- tb3_waffle_stereo.urdf.xacro
29 |   |   '-- launch/
30 |   |       '-- spawn_robot.launch.py

```

```

31 | |
32 |   '-- turtlebot3_simulations/      # Worlds Gazebo personnalisées
33 |     '-- worlds/
34 |         |-- corridor.world          # Scenario A
35 |         |-- open_space.world        # Scenario B
36 |         '-- warehouse.world         # Scenario C
37 |-- install/                          # Artefacts colcon (genere)
38 |-- build/                            # Binaires (genere)
39 '-- log/                              # Journaux de build

```

Listing 5.3 – Structure du workspace colcon

5.3.2 Configuration de la caméra stéréo sur le Turtlebot3 Waffle

Le modèle Turtlebot3 Waffle intègre nativement une caméra RGB (caméra Intel RealSense R200 simulée). Pour ce travail, une seconde caméra est ajoutée via le fichier URDF/Xacro afin de constituer la paire stéréo :

```

1 <!-- Liaison fixe entre le châssis et la caméra droite -->
2 <!-- Baseline = 0.12 m selon l'axe_Y (inter-oculaire) -->
3 <joint name="camera_right_joint" type="fixed">
4   <parent link="base_link"/>
5   <child link="camera_right_link"/>
6   <origin xyz="-0.064 -0.06 0.120" rpy="0 0 0"/>
7 </joint>
8
9 <link name="camera_right_link">
10  <visual>
11    <geometry><box size="0.015 0.030 0.027"/></geometry>
12    <material name="dark_grey"/>
13  </visual>
14 </link>
15
16 <!-- Plugin Gazebo pour la caméra droite -->
17 <gazebo reference="camera_right_link">
18   <sensor type="camera" name="stereo_camera_right">
19     <update_rate>30</update_rate>
20     <camera>
21       <horizontal_fov>1.3962634</horizontal_fov>
22       <image>
23         <width>640</width> <height>480</height>
24         <format>R8G8B8</format>
25       </image>
26       <clip><near>0.02</near><far>300</far></clip>

```

```

27 <camera>
28 <plugin_name="camera_right_controller"
29 filename="libgazebo_ros_camera.so">
30 <ros>
31 <namespace>/stereo</namespace>
32 <remapping>image_raw:=right/image_raw</remapping>
33 <remapping>camera_info:=right/camera_info</remapping>
34 </ros>
35 <frame_name>camera_right_link</frame_name>
36 </plugin>
37 </sensor>
38 </gazebo>

```

Listing 5.4 – Extension URDF : ajout de la caméra stéréo droite (extrait de `tb3_waffle_stereo.urdf.xacro`)

La caméra gauche (`camera_link`, native au Waffle) publie sur `/stereo/left/image_raw` et la caméra droite ajoutée publie sur `/stereo/right/image_raw`. Les deux caméras partagent la même focale simulée ($f = 462,1$ px) et sont séparées par une baseline de $B = 0,12$ m, ce qui permet l'estimation de la profondeur entre 0,3 m et 10 m dans les environnements intérieurs.

5.4 Architecture des Nœuds ROS 2

5.4.1 Vue d'ensemble du graphe de communication

L'architecture logicielle du système est organisée en quatre couches fonctionnelles communicantes via des topics ROS 2 (modèle publication/abonnement DDS) :

1. **Couche capteurs** : Production des flux de données brutes (images stéréo, odométrie, LiDAR simulé).
2. **Couche perception** : Traitement des images stéréo, génération de la carte de disparité/profondeur, détection d'obstacles.
3. **Couche navigation** : Prise de décision par l'agent PPO, calcul de la récompense, gestion de l'objectif.
4. **Couche supervision** : Visualisation RViz2, enregistrement des données (`ros2 bag`), journalisation des métriques.

5.4.2 Description des nœuds principaux

TABLE 5.3 – Nœuds ROS 2 du système et leurs rôles

Nœud	Package	Fréquence	Rôle
/psmnet_node	stereo_vision	10 Hz	Inférence PSMNet, publication carte de disparité/profondeur
/stereo_proc	stereo_image_proc	30 Hz	Rectification des images stéréo
/obstacle_detector	stereo_vision	10 Hz	RANSAC sol, DBSCAN obstacles, grille d'occupation
/rl_agent_node	rl_navigation	10 Hz	Politique PPO, commande /cmd_vel
/reward_node	rl_navigation	10 Hz	Calcul récompense, détection collision, gestion goal
/goal_manager	rl_navigation	événem.	Publication de l'objectif, détection d'arrivée
/tf2_broadcaster	turtlebot3_stereo	50 Hz	Transformations : base_link → caméras → odom → map
/rviz2	rviz2	25 Hz	Visualisation : images, disparité, marqueurs 3D, trajectoire

5.4.3 Topics ROS 2 principaux

TABLE 5.4 – Topics ROS 2 du système de perception-navigation

Topic	Type	Hz	Flux
/stereo/left/image_raw	sensor_msgs/Image	30	caméra gauche → stereo_proc
/stereo/right/image_raw	sensor_msgs/Image	30	caméra droite → stereo_proc
/stereo/left/image_rect	sensor_msgs/Image	30	stereo_proc → psmnet_node
/stereo/right/image_rect	sensor_msgs/Image	30	stereo_proc → psmnet_node
/stereo/disparity	stereo_msgs/DisparityImage	10	psmnet_node → RViz2
/stereo/depth	sensor_msgs/Image	10	psmnet_node → obstacle_detector, rl_agent
/obstacles/markers	visualization_msgs/MarkerArray	10	obstacle_detector → RViz2
/obstacles/grid	nav_msgs/OccupancyGrid	10	obstacle_detector → rl_agent
/odom	nav_msgs/Odometry	30	Gazebo → rl_agent, reward_node
/scan	sensor_msgs/LaserScan	5	LiDAR simulé → reward_node
/cmd_vel	geometry_msgs/Twist	10	rl_agent → Turtlebot3
/reward	std_msgs/Float32	10	reward_node → rl_agent
/collision	std_msgs/Bool	10	reward_node → rl_agent
/goal_pose	geometry_msgs/PoseStamped	événement	goal_manager → rl_agent, reward_node

5.4.4 Nœud PSMNet sous ROS 2 Humble

La différence principale avec ROS 1 Noetic réside dans l'API de création des nœuds et l'utilisation de `rclpy` à la place de `rospy`, et dans le mécanisme d'exécution par `rclpy.spin()` :

```

1  #!/usr/bin/env python3
2  """psmnet_node.py      Noeud ROS2 Humble pour l'inference PSMNet."""
3
4  import rclpy
5  from rclpy.node import Node
6  from sensor_msgs.msg import Image, CameraInfo
7  from stereo_msgs.msg import DisparityImage
8  from cv_bridge import CvBridge
9  from message_filters import ApproximateTimeSynchronizer, Subscriber
10 import torch, numpy as np, cv2
11
12 class PSMNetNode(Node):
13
14     def __init__(self):
15         super().__init__('psmnet_node')
16
17         # Parametres declarables depuis la CLI ou un fichier YAML
18         self.declare_parameter('model_path', 'checkpoints/
19                                psmnet_best.pth')
20         self.declare_parameter('focal', 462.1)
21         self.declare_parameter('baseline', 0.12)
22         self.declare_parameter('maxdisp', 192)
23
24         model_path = self.get_parameter('model_path').value
25         self.focal = self.get_parameter('focal').value
26         self.baseline = self.get_parameter('baseline').value
27         self.maxdisp = self.get_parameter('maxdisp').value
28
29         # Charger le modele PSMNet
30         from models.psmnet import PSMNet
31         self.device = torch.device('cuda' if torch.cuda.is_available
32                                    ()
33                                    else 'cpu')
34         self.model = PSMNet(maxdisp=self.maxdisp).to(self.device)
35         self.model.load_state_dict(
36             torch.load(model_path, map_location=self.device))
37         self.model.eval()
38         self.bridge = CvBridge()
39
40         # Abonnements avec synchronisation temporelle (tolerance 30
41         ms)
42         sub_l = Subscriber(self, Image, '/stereo/left/image_rect')

```

```

40     sub_r = Subscriber(self, Image, '/stereo/right/image_rect')
41     self.sync = ApproximateTimeSynchronizer(
42         [sub_l, sub_r], queue_size=5, slop=0.03)
43     self.sync.registerCallback(self.stereo_callback)
44
45     # Publications
46     self.pub_disp = self.create_publisher(
47         DisparityImage, '/stereo/disparity', 1)
48     self.pub_depth = self.create_publisher(
49         Image, '/stereo/depth', 1)
50
51     self.get_logger().info(
52         f'PSMNet pret      focal={self.focal:.1f}px, '
53         f'baseline={self.baseline:.3f}m, device={self.device}')
54
55     def stereo_callback(self, msg_l, msg_r):
56         left = self.bridge.imgmsg_to_cv2(msg_l, 'rgb8')
57         right = self.bridge.imgmsg_to_cv2(msg_r, 'rgb8')
58
59         # Pretraitement : redimensionner et normaliser
60         left_t = self._preprocess(left)
61         right_t = self._preprocess(right)
62
63         with torch.no_grad():
64             disp = self.model(left_t, right_t) # (1,1,H,W)
65
66         disp_np = disp.squeeze().cpu().numpy()
67
68         # Conversion disparite -> profondeur
69         depth = np.where(disp_np > 0.1,
70                         (self.focal * self.baseline) / disp_np,
71                         0.0).astype(np.float32)
72         depth = cv2.medianBlur(depth, 5)
73
74         # Publication
75         depth_msg = self.bridge.cv2_to_imgmsg(depth, '32FC1')
76         depth_msg.header = msg_l.header
77         self.pub_depth.publish(depth_msg)
78
79     def _preprocess(self, img):
80         img = cv2.resize(img, (512, 256))
81         t = torch.from_numpy(img).permute(2,0,1).float() / 255.0
82         mean = torch.tensor([0.485, 0.456, 0.406]).view(3,1,1)

```

```

83         std = torch.tensor([0.229, 0.224, 0.225]).view(3,1,1)
84         return ((t - mean) / std).unsqueeze(0).to(self.device)
85
86 def main(args=None):
87     rclpy.init(args=args)
88     node = PSMNetNode()
89     rclpy.spin(node)
90     node.destroy_node()
91     rclpy.shutdown()
92
93 if __name__ == '__main__':
94     main()

```

Listing 5.5 – Nœud psmnet_node.py — Inférence PSMNet sous ROS 2 Humble

5.4.5 Nœud de l’agent PPO sous ROS 2 Humble

La communication entre l’environnement Gymnasium et ROS 2 Humble est gérée par le nœud `rl_agent_node.py`. L’environnement hérite de `gymnasium.Env` et utilise les abonnements/publications ROS 2 via un nœud séparé exécuté dans un thread dédié :

```

1  #!/usr/bin/env python3
2  """gym_env.py — Environnement Gymnasium pour ROS 2 Humble."""
3
4  import rclpy
5  from rclpy.node import Node
6  import gymnasium as gym
7  import numpy as np
8  import threading
9  from geometry_msgs.msg import Twist
10 from nav_msgs.msg import Odometry
11 from sensor_msgs.msg import Image
12 from std_msgs.msg import Bool, Empty
13 from cv_bridge import CvBridge
14
15 class TurtlebotStereoEnv(gym.Env):
16     """Environnement de navigation stereo pour Turtlebot3 Waffle."""
17
18     def __init__(self):
19         # Initialiser ROS 2 dans un thread separ
20         rclpy.init()
21         self._node = rclpy.create_node('rl_gym_env')
22         self._executor = rclpy.executors.MultiThreadedExecutor()
23         self._executor.add_node(self._node)

```

```

24     self._spin_thread = threading.Thread(
25         target=self._executor.spin, daemon=True)
26     self._spin_thread.start()
27
28     self.bridge = CvBridge()
29     self._depth = None
30     self._odom = None
31     self._collided = False
32
33     # Abonnements
34     self._node.create_subscription(
35         Image, '/stereo/depth', self._depth_cb, 1)
36     self._node.create_subscription(
37         Odometry, '/odom', self._odom_cb, 1)
38     self._node.create_subscription(
39         Bool, '/collision', self._coll_cb, 1)
40
41     # Publications
42     self._cmd_pub = self._node.create_publisher(
43         Twist, '/cmd_vel', 1)
44     self._reset_pub = self._node.create_publisher(
45         Empty, '/reset_simulation', 1)
46
47     # Espaces Gymnasium
48     self.observation_space = gym.spaces.Dict({
49         'depth': gym.spaces.Box(
50             0, 10, (64, 64), dtype=np.float32),
51         'goal': gym.spaces.Box(
52             -10, 10, (3,), dtype=np.float32),
53         'vel': gym.spaces.Box(
54             -2, 2, (2,), dtype=np.float32),
55     })
56     self.action_space = gym.spaces.Box(
57         low = np.array([0.0, -1.0]),
58         high = np.array([0.5, 1.0]),
59         dtype=np.float32)
60
61     self._goal = np.array([3.0, 0.0, 0.0])
62
63     def step(self, action):
64         v, omega = float(action[0]), float(action[1])
65         msg = Twist()
66         msg.linear.x = v

```

```

67     msg.angular.z = omega
68     self._cmd_pub.publish(msg)
69     import time; time.sleep(0.1)    # 10 Hz
70
71     obs  = self._get_obs()
72     done = self._collided or self._goal_reached()
73     rew  = self._compute_reward()
74     return obs, rew, done, False, {}
75
76     def reset(self, *, seed=None, options=None):
77         self._reset_pub.publish(Empty())
78         import time; time.sleep(0.5)
79         self._collided = False
80         self._goal = self._sample_goal()
81         return self._get_obs(), {}
82
83     # ... (m thodes _get_obs, _compute_reward, _goal_reached
      in changes)

```

Listing 5.6 – Environnement Gymnasium connecté à ROS 2 Humble (extrait)

5.4.6 Fichier de lancement principal (launch.py)

Sous ROS 2 Humble, les fichiers de lancement sont écrits en Python (.launch.py) et non plus en XML :

```

1  """full_stereo_nav.launch.py    Lance le systeme complet sous ROS 2
   Humble."""
2
3  from launch import LaunchDescription
4  from launch.actions import IncludeLaunchDescription,
   DeclareLaunchArgument
5  from launch.launch_description_sources import
   PythonLaunchDescriptionSource
6  from launch_ros.actions import Node
7  from launch.substitutions import LaunchConfiguration,
   PathJoinSubstitution
8  from launch_ros.substitutions import FindPackageShare
9
10 def generate_launch_description():
11
12     world_arg = DeclareLaunchArgument(
13         'world', default_value='corridor',

```

```

14         description='Monde_Gazebo:corridor|open_space|
15             warehouse')
16
17 world = LaunchConfiguration('world')
18
19 # 1. Gazebo Classic avec le monde choisi
20 gazebo = IncludeLaunchDescription(
21     PythonLaunchDescriptionSource([
22         FindPackageShare('gazebo_ros'), '/launch/gazebo.launch.
23             py']),
24     launch_arguments={
25         'world': PathJoinSubstitution([
26             FindPackageShare('turtlebot3_stereo'),
27             'worlds', [world, '.world']]),
28         'verbose': 'false'}.items())
29
30 # 2. Spawner du robot Turtlebot3 Waffle stereo
31 spawn_robot = IncludeLaunchDescription(
32     PythonLaunchDescriptionSource([
33         FindPackageShare('turtlebot3_stereo'),
34         '/launch/spawn_robot.launch.py']))
35
36 # 3. Rectification stereo (remplace stereo_image_proc ROS 1)
37 stereo_proc = Node(
38     package='stereo_image_proc',
39     executable='stereo_image_proc',
40     namespace='stereo',
41     parameters=[{'approximate_sync': True, 'queue_size': 5}])
42
43 # 4. Noeud PSMNet
44 psmnet = Node(
45     package='stereo_vision',
46     executable='psmnet_node',
47     name='psmnet_node',
48     parameters=[{
49         'model_path': 'checkpoints/psmnet_best.pth',
50         'focal': 462.1,
51         'baseline': 0.12,
52         'maxdisp': 192,
53     }],
54     output='screen')
55
56 # 5. Noeud de detection d'obstacles
57 obstacle_det = Node(

```

```

55     package='stereo_vision',
56     executable='obstacle_detector',
57     name='obstacle_detector',
58     parameters=[{
59         'ransac_thresh': 0.02,
60         'dbscan_eps': 0.30,
61         'dbscan_min_pts': 10,
62         'danger_dist': 0.50,
63     }],
64     output='screen')
65
66 # 6. Noeud agent RL (mode deployment)
67 rl_agent = Node(
68     package='rl_navigation',
69     executable='rl_agent_node',
70     name='rl_agent_node',
71     parameters=[{
72         'model_path': 'models/ppo_best.zip',
73         'deterministic': True,
74     }],
75     output='screen')
76
77 # 7. RViz2 avec configuration personnalisée
78 rviz = Node(
79     package='rviz2',
80     executable='rviz2',
81     name='rviz2',
82     arguments=['-d', PathJoinSubstitution([
83         FindPackageShare('stereo_vision'),
84         'rviz', 'stereo_nav.rviz'])],
85     output='screen')
86
87 return LaunchDescription([
88     world_arg,
89     gazebo, spawn_robot,
90     stereo_proc, psmnet, obstacle_det,
91     rl_agent, rviz,
92 ])

```

Listing 5.7 – Fichier full_stereo_nav.launch.py — Lancement complet du système

5.5 Configuration de la Simulation Gazebo

5.5.1 Mondes Gazebo personnalisés

Trois environnements de simulation ont été créés pour couvrir la diversité des conditions de navigation rencontrées en environnements intérieurs :

Scénario A : Couloir étroit (`corridor.world`)

Ce monde représente un couloir rectiligne de 10 m de long et 2,5 m de large, parsemé d'obstacles statiques (boîtes, cylindres) disposés en quinconce. Ce scénario évalue la capacité du robot à naviguer dans un espace contraint et à planifier des trajectoires évitant des obstacles à faible marge de manœuvre. Les dimensions sont inspirées des couloirs de bureaux ou d'entrepôts, ce qui reflète le contexte d'application du Turtlebot3.

Scénario B : Espace ouvert avec obstacles mobiles (`open_space.world`)

Un espace de 8 m × 8 m avec des obstacles statiques (murs, caisses) et des obstacles dynamiques (un acteur Gazebo simulant un opérateur humain se déplaçant à 0,5 m/s selon une trajectoire sinusoïdale). Ce scénario évalue la réactivité de l'agent face à des obstacles non prévisibles, représentative des conditions réelles d'un robot partageant son espace avec des personnes.

Scénario C : Entrepôt complexe (`warehouse.world`)

Un espace de 12 m × 8 m reproduisant un entrepôt avec des étagères (surfaces verticales lisses, challenging pour la perception stéréo), des allées étroites et un mélange d'obstacles statiques et dynamiques. Ce scénario est le plus exigeant : il combine les difficultés géométriques (surfaces sans texture propices aux erreurs de disparité) et cinématiques (obstacles mobiles).

5.5.2 Configuration RViz2 pour la supervision

RViz2 est configuré pour afficher simultanément les données clés du pipeline :

```

1 # stereo_nav.rviz      Configuration de visualisation pour ROS 2
   Humble
2
3 Panels:
4   - Class: rviz_common/Displays
5   - Class: rviz_common/Selection
6   - Class: rviz_common/Time
7
8 Visualization Manager:
9   Displays:
10    # Images stereo rectifiees

```

```

11 - Class: rviz_default_plugins/Image
12   Topic:  /stereo/left/image_rect
13   Name:   "Image gauche rectifiee"
14
15 # Carte de disparite (fausses couleurs plasma)
16 - Class: rviz_default_plugins/Image
17   Topic:  /stereo/disparity
18   Name:   "Disparite PSMNet"
19
20 # Nuage de points obstacles (obstacles uniquement, sans le sol)
21 - Class: rviz_default_plugins/PointCloud2
22   Topic:  /obstacles/pointcloud
23   Name:   "Obstacles 3D"
24   Color:  255 50 50 # rouge
25
26 # Boites englobantes 3D
27 - Class: rviz_default_plugins/MarkerArray
28   Topic:  /obstacles/markers
29   Name:   "Bounding Boxes"
30
31 # Grille d'occupation locale
32 - Class: rviz_default_plugins/Map
33   Topic:  /obstacles/grid
34   Name:   "Carte d'occupation"
35
36 # Modele 3D du robot
37 - Class: rviz_default_plugins/RobotModel
38   Description_Topic: /robot_description
39   Name:   "Turtlebot3 Waffle"
40
41 # Transformations TF2
42 - Class: rviz_default_plugins/TF
43   Name: "Arbre TF2"

```

Listing 5.8 – Configuration RViz2 (stereo_nav.rviz) — panneaux de visualisation

5.5.3 Commandes de lancement de la simulation

```

1 # Terminal 1      Environnement complet (Gazebo + perception +
   navigation + RViz2)
2 source ~/stereo_nav_ws/install/setup.bash
3 ros2 launch stereo_vision full_stereo_nav.launch.py world:=corridor
4

```

```

5 #
6 # Terminal 2 (optionnel)      Monitoring des topics en temps reel
7 ros2 topic hz /stereo/depth    # Verifier 10 Hz
8 ros2 topic hz /cmd_vel         # Verifier 10 Hz
9 ros2 topic echo /reward        # Surveiller les recompenses
10
11 # Terminal 3 (optionnel)      Graph des noeuds actifs
12 rqt_graph
13
14 # Terminal 4 (optionnel)      Enregistrement pour analyse posterieure
15 ros2 bag record /stereo/depth /stereo/disparity \
16                        /obstacles/markers /cmd_vel /odom \
17                        /reward /collision \
18                        -o ~/bags/experiment_$(date +%Y%m%d_%H%M%S)

```

Listing 5.9 – Séquence de commandes pour démarrer la simulation complète

5.6 Méthodologie d’Évaluation

5.6.1 Protocole expérimental

L’évaluation du système complet porte sur **50 trajectoires de test** réparties équitablement sur les trois scénarios :

- **Scénario A (Couloir)** : 15 trajectoires, positions de départ et d’objectif tirées aléatoirement selon une graine fixée ($seed = 0$ à 14).
- **Scénario B (Espace ouvert)** : 20 trajectoires (10 avec obstacles statiques seuls, 10 avec obstacles mobiles ajoutés).
- **Scénario C (Entrepôt)** : 15 trajectoires, les plus complexes en termes de géométrie et d’obstacles dynamiques.

Pour chaque trajectoire, le robot part d’une position initiale fixée et doit atteindre un objectif à une distance euclidienne d’environ 4 m en moins de 500 pas de décision (soit 50 s à 10 Hz). Une trajectoire est déclarée :

- **Réussie** si le robot atteint l’objectif ($d < 0,3$ m) sans collision.
- **Échouée pour collision** si le robot entre en contact avec un obstacle (détecté par le topic /collision).
- **Échouée par timeout** si la limite de 500 pas est dépassée sans atteindre l’objectif.

5.6.2 Métriques d'évaluation du système complet

TABLE 5.5 – Métriques d'évaluation du système de navigation autonome

Métrique	Définition	Cible
Taux de succès (%)	$\frac{\text{trajectoires réussies}}{\text{trajectoires totales}} \times 100$	$> 85 \%$
Taux de collision (%)	$\frac{\text{trajectoires avec collision}}{\text{trajectoires totales}} \times 100$	$< 5 \%$
EPE disparité (px)	$\frac{1}{ \mathcal{M} } \hat{d} - d^* $ sur pixels valides	$< 1,5 \text{ px}$
D1-all (%)	% pixels avec $\text{err} > 3 \text{ px}$ et $> 5 \%$ de d^*	$< 3,5 \%$
Longueur de trajectoire	Ratio $\frac{L_{\text{robot}}}{L_{\text{optimal}}}$ (trajectoire normalisée)	$< 1,5$
Latence pipeline (ms)	Délai acquisition $\rightarrow$ publication / cmd_vel	$< 200 \text{ ms}$
Latence PSMNet (ms)	Délai images stéréo $\rightarrow$ carte de profondeur	$< 100 \text{ ms}$
Récompense cumulée	$G_t = \sum_{k=0}^T \gamma^k r_{tk}$ par épisode	Convergence stable

5.6.3 Comparaison avec la baseline classique

Chaque métrique est comparée à une baseline SGM + planificateur A* classique :

- **Perception baseline** : Algorithme SGM d'OpenCV (StereoSGBM, paramètres optimisés) à la place de PSMNet.
- **Navigation baseline** : Planificateur de chemin A* avec carte d'occupation statique et évitement DWA (*Dynamic Window Approach*), configuré via Nav2 sous ROS 2 Humble.

Cette comparaison permet de quantifier précisément l'apport de l'apprentissage profond (PSMNet vs SGM) et de l'apprentissage par renforcement (PPO vs A*+DWA) sur les performances globales du système.

5.6.4 Script d'évaluation automatisée

```

1  #!/usr/bin/env python3
2  """evaluate_system.py    Evaluation automatisée du système complet.
   """
3
4  import rclpy, numpy as np, json, time
5  from stable_baselines3 import PPO
6  from gym_env import TurtlebotStereoEnv
7
8  def evaluate(model_path, n_episodes=50, seeds=None):

```

```

9     env    = TurtlebotStereoEnv()
10    model = PPO.load(model_path, env=env)
11
12    results = {'success': [], 'collision': [], 'timeout': [],
13              'length': [], 'reward': [], 'latency_ms': []}
14
15    seeds = seeds or list(range(n_episodes))
16
17    for ep, seed in enumerate(seeds):
18        obs, _ = env.reset(seed=seed)
19        done, steps, cumrew = False, 0, 0.0
20        t0 = time.perf_counter()
21
22        while not done:
23            action, _ = model.predict(obs, deterministic=True)
24            obs, rew, done, truncated, info = env.step(action)
25            cumrew += rew; steps += 1
26            done = done or truncated or (steps >= 500)
27
28        elapsed_ms = (time.perf_counter() - t0) * 1000 / steps
29
30        results['success'].append(info.get('goal_reached', False))
31        results['collision'].append(info.get('collided', False))
32        results['timeout'].append(steps >= 500)
33        results['length'].append(steps)
34        results['reward'].append(cumrew)
35        results['latency_ms'].append(elapsed_ms)
36
37        print(f"__Episode_{ep+1:2d}/50__"
38              f"{'OK' if info.get('goal_reached') else 'FAIL'}__")
39        f"steps={steps}__rew={cumrew:.1f}__"
40        f"lat={elapsed_ms:.1f}ms")
41
42    # Synthèse finale
43    print("\n" + "="*60)
44    print(f"Taux de succes__ : {np.mean(results['success'])*100:.1f}__%")
45    print(f"Taux de collision__ : {np.mean(results['collision'])*100:.1f}__%")
46    print(f"Taux de timeout__ : {np.mean(results['timeout'])*100:.1f}__%")
47    print(f"Longueur moyenne__ : {np.mean(results['length']):.1f}__pas")

```

```

48     print(f"Recompense_moy. : {np.mean(results['reward']):.1f}")
49     print(f"Latence_PPO_moy. : {np.mean(results['latency_ms']):.1f}
      ms")
50
51     # Sauvegarde JSON
52     with open('evaluation_results.json', 'w') as f:
53         json.dump({k: list(map(float, v))
54                     for k, v in results.items()}, f, indent=2)
55     print("Resultats sauvegardes -> evaluation_results.json")
56     env.close()
57
58 if __name__ == '__main__':
59     evaluate('models/ppo_best.zip', n_episodes=50)

```

Listing 5.10 – Script d'évaluation automatisée sur 50 trajectoires

5.7 Discussion

5.7.1 Choix de ROS 2 Humble vs ROS 1 Noetic

Le passage de ROS 1 Noetic à ROS 2 Humble implique plusieurs adaptations architecturales :

- **API Python** : `rospy` → `rclpy`. Les nœuds héritent désormais de `rclpy.node.Node` et sont initialisés via `rclpy.init()`.
- **Fichiers de lancement** : Les fichiers XML (`.launch`) sont remplacés par des fichiers Python (`.launch.py`), plus flexibles et intégrables avec la logique Python.
- **Communication** : ROS 2 utilise DDS (*Data Distribution Service*) sans nœud maître central, ce qui améliore la robustesse aux pannes. La synchronisation temporelle reste assurée par `message_filters.ApproximateTimeSynchronizer`, disponible en ROS 2.
- **Build system** : `catkin_make` → `colcon build`. Les packages Python utilisent `setup.py` et `ament_python`.
- **Paramètres** : Les paramètres ROS 2 sont déclarés explicitement via `declare_parameter()` et récupérés via `get_parameter()`.

5.7.2 Gazebo Classic vs Gazebo Ignition

Sous ROS 2 Humble sur Ubuntu 22.04, Gazebo Classic 11 est préféré à Gazebo Ignition (Fortress) pour les raisons suivantes :

1. Les packages Turtlebot3 (`turtlebot3_simulations`, branche `humble`) sont testés et validés avec Gazebo Classic.
2. Le plugin `libgazebo_ros_camera.so` est mieux documenté et plus stable que son équivalent Ignition pour la simulation de caméras stéréo.

3. Gazebo Ignition présente des bugs connus liés à la génération du topic `/cmd_vel` lors de l'utilisation de ses plugins différentiels, ce qui peut bloquer la navigation [?].

5.8 Conclusion

Ce chapitre a présenté en détail l'intégration du système de vision stéréoscopique et de navigation autonome au sein de l'environnement ROS 2 Humble / Gazebo 11 / RViz2. La configuration présentée — workspace colcon structuré, nœuds Python `rclpy`, launch files Python, URDF Turtlebot3 Waffle avec caméra stéréo simulée, et trois mondes Gazebo personnalisés — constitue une infrastructure expérimentale complète et reproductible.

Les principales contributions de ce chapitre sont : (1) l'adaptation de l'ensemble du pipeline ROS 1 vers ROS 2 Humble avec ses spécificités (DDS, `rclpy`, `colcon`, launch Python); (2) la définition d'un protocole d'évaluation rigoureux sur 50 trajectoires et trois scénarios de complexité croissante; et (3) l'outillage complet de supervision (RViz2, `ros2 bag`, script d'évaluation automatisée). Les résultats quantitatifs obtenus sur ce dispositif expérimental sont présentés et analysés dans le Chapitre 6.

Chapitre 6

Résultats et Analyse

Ce chapitre est à compléter. Il couvrira : les métriques quantitatives (EPE, D1-all, taux de succès de navigation, taux de collision, latence), les visualisations qualitatives (cartes de disparité, nuages de points, trajectoires), et l'analyse comparative par rapport aux baselines classiques (SGM + planificateur A).*

Chapitre 7

Conclusion et Perspectives

Ce chapitre est à compléter. Il résumera les conclusions principales, discutera des limitations de l'approche actuelle (sim-to-real gap, latence résiduelle, robustesse aux occultations) et proposera des pistes de recherche futures (modèles Foundation, RAFT-Stereo, déploiement sur robot réel).

Bibliographie

- [1] R. Lee, J. Usaquen et F. Martínez, “Développement de systèmes de vision stéréoscopique à faible coût pour la robotique éducative : un état de l’art,” *Global Journal of Engineering and Technology Advances*, 2024.
- [2] S. Badrloo, “Détection d’obstacles rapide et précise basée sur la vision stéréo et l’apprentissage profond,” *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 2025.
- [3] M. Patel, F. Yang, Y. Qiu, C. Cadena, S. Scherer, M. Hutter et W. Wang, “TartanGround : un jeu de données à grande échelle pour la perception et la navigation des robots de sol,” *arXiv preprint arXiv :2505.10696*, 2025.
- [4] J.-R. Chang et Y.-S. Chen, “Pyramid stereo matching network,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5410–5418, 2018.
- [5] L. Lipson, Z. Teed et J. Deng, “RAFT-Stereo : transformées de champ itératif multi-niveaux pour la correspondance stéréo,” in *International Conference on 3D Vision (3DV)*, 2021.
- [6] N. Mayer *et al.*, “Un grand jeu de données pour entraîner des réseaux convolutifs pour la disparité, le flux optique et l’estimation de flux de scène,” in *CVPR*, 2016.
- [7] A. Geiger, P. Lenz, C. Stiller et R. Urtasun, “La vision rencontre la robotique : le jeu de données KITTI,” *The International Journal of Robotics Research*, vol. 32, n° 11, pp. 1231–1237, 2013.
- [8] D. Scharstein *et al.*, “Jeux de données stéréo haute résolution avec vérité terrain sub-pixel,” in *German Conference on Pattern Recognition (GCPR)*, 2014.
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford et O. Klimov, “Algorithmes d’optimisation par politique proximale,” *arXiv preprint arXiv :1707.06347*, 2017.
- [10] M. Quigley *et al.*, “ROS : un système d’exploitation robotique open-source,” in *ICRA Workshop on Open Source Software*, vol. 3, n° 3.2, 2009.
- [11] S. Gollapudi, *Apprendre la Vision par Ordinateur avec OpenCV : avec les CNN et les RNN en Apprentissage Profond*. Apress, 2019.